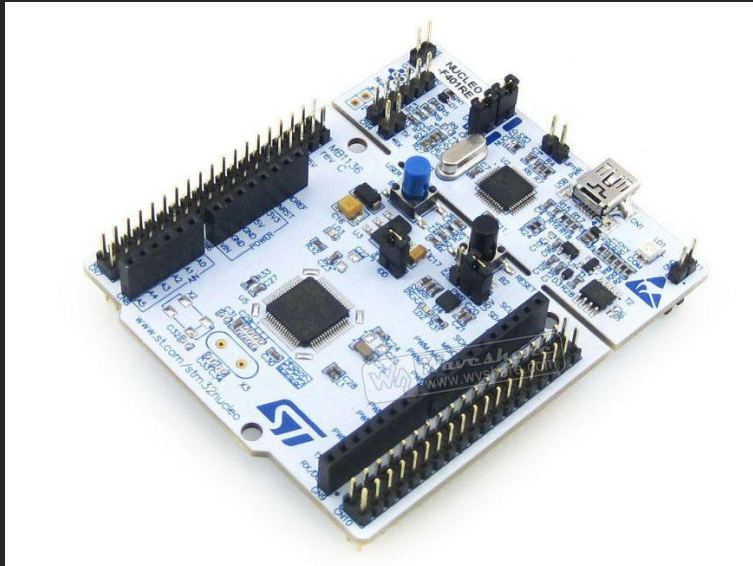
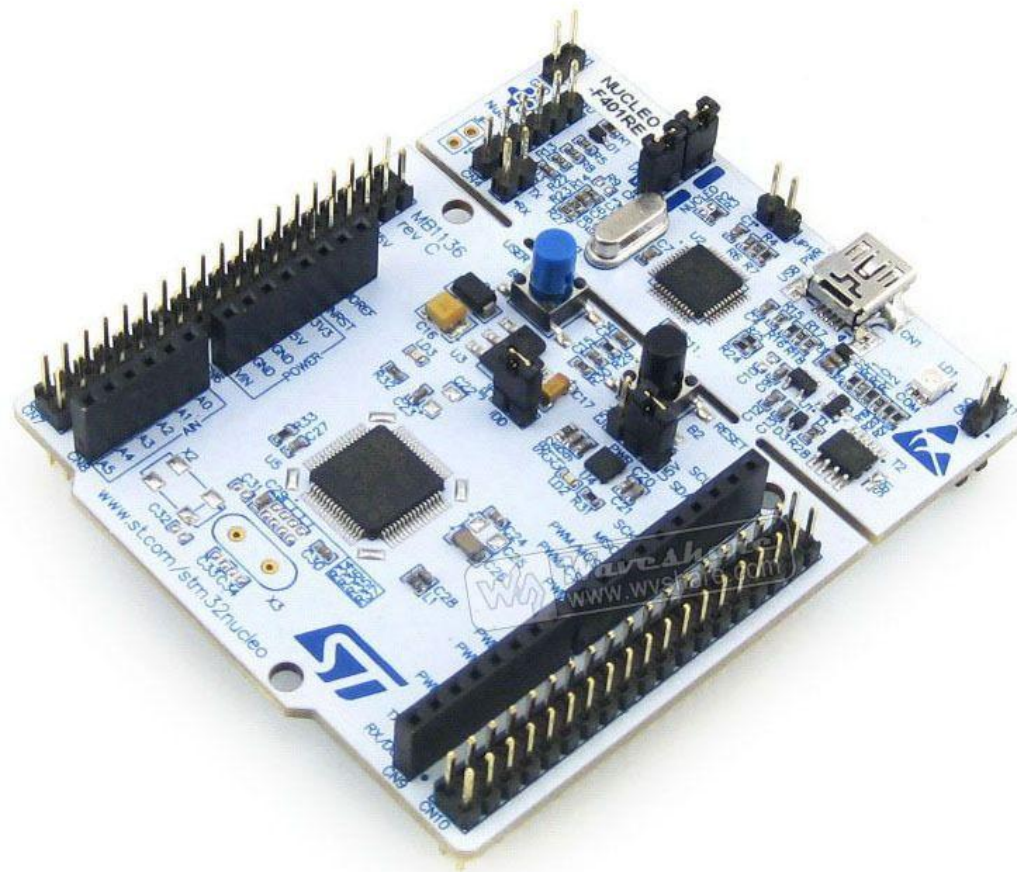


STM32F411 Nucleo Board

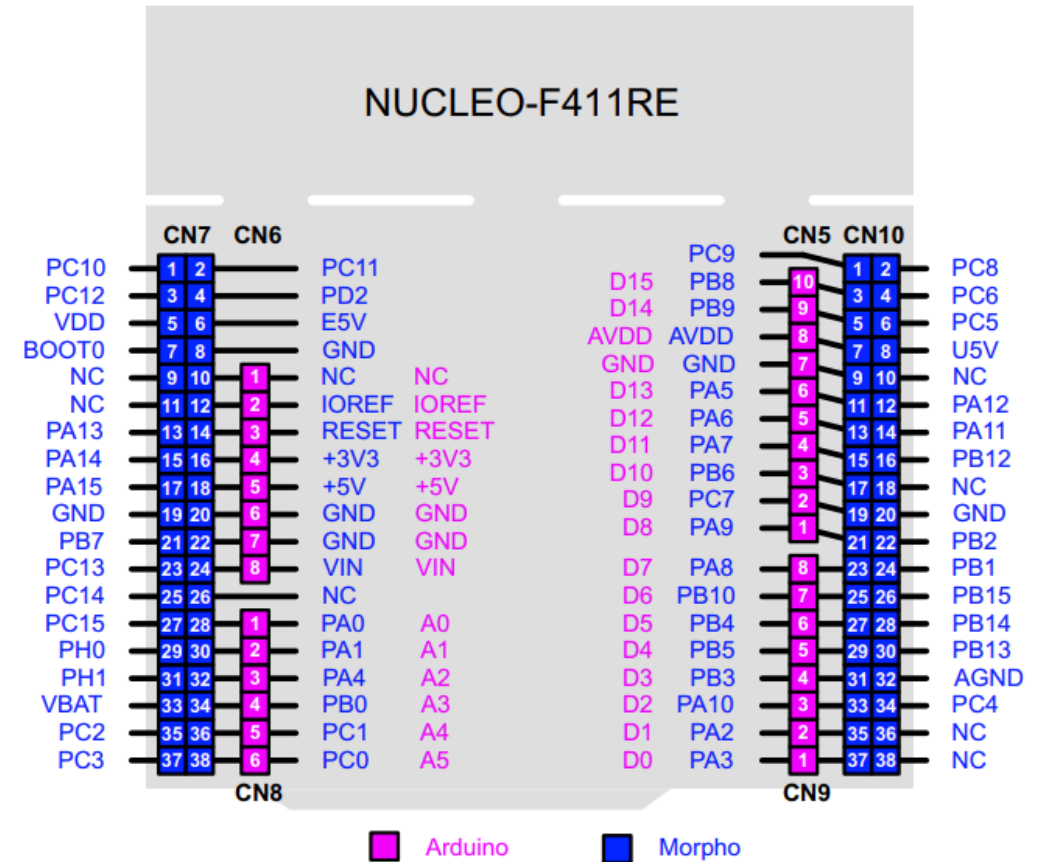
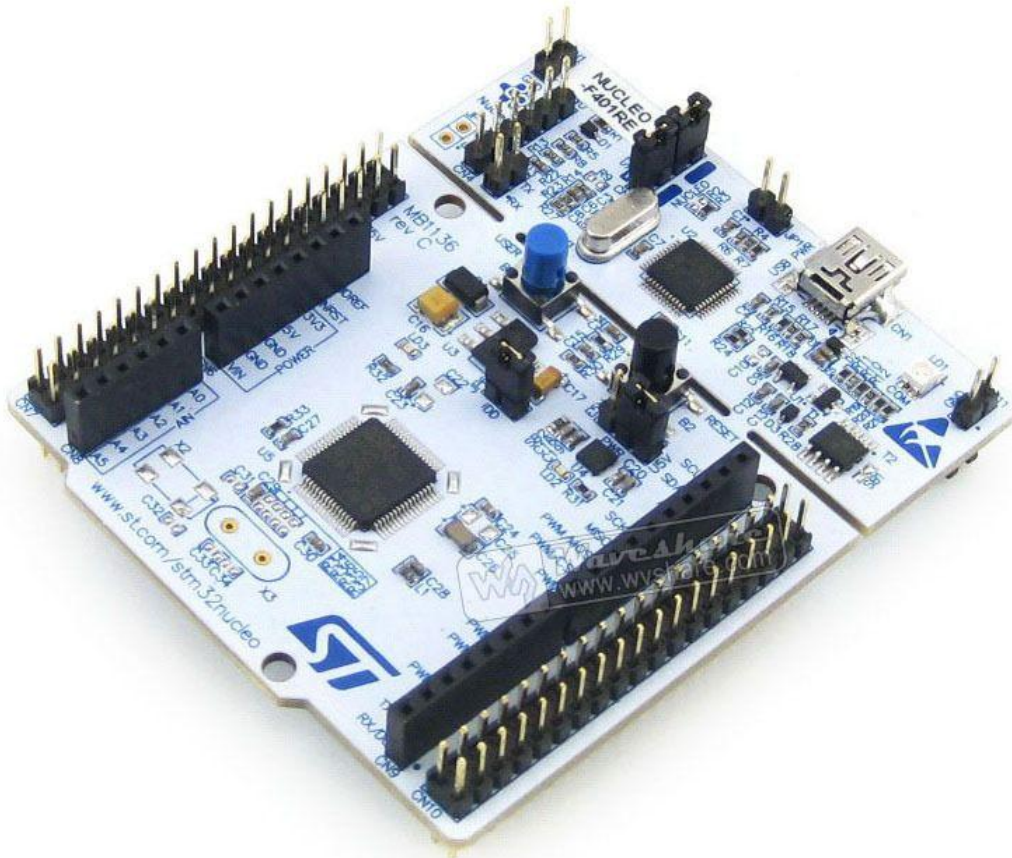


ATHANASIADIS ANGELOS
KARANASSOS DIMITRIOS

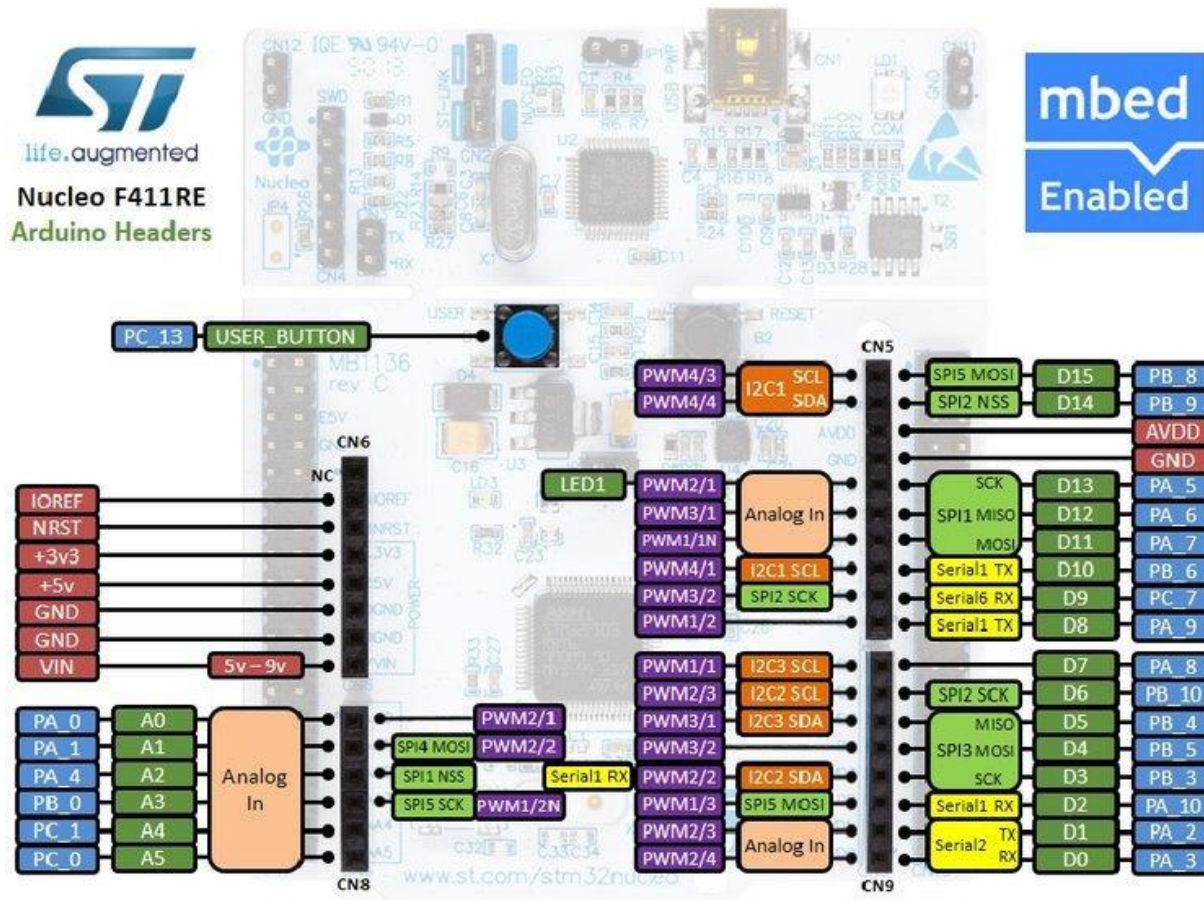
STM32F411 Nucleo Board



Board IO Pins Arduino Compatible



Nucleo Board Pinouts



Labels usable in code

PX_Y MCU pin without conflict

PX_Y MCU pin connected to other components

See [PeripheralPins.c](#) (link below) for more information

Labels not usable in code (for information only)

XXX Serial pins (USART/UART)

XXX SPI pins

XXX I2C pins

XXX PWMOut pins (TIMER n/c[N])
n = Timer number c = Channel
N = Inverted channel

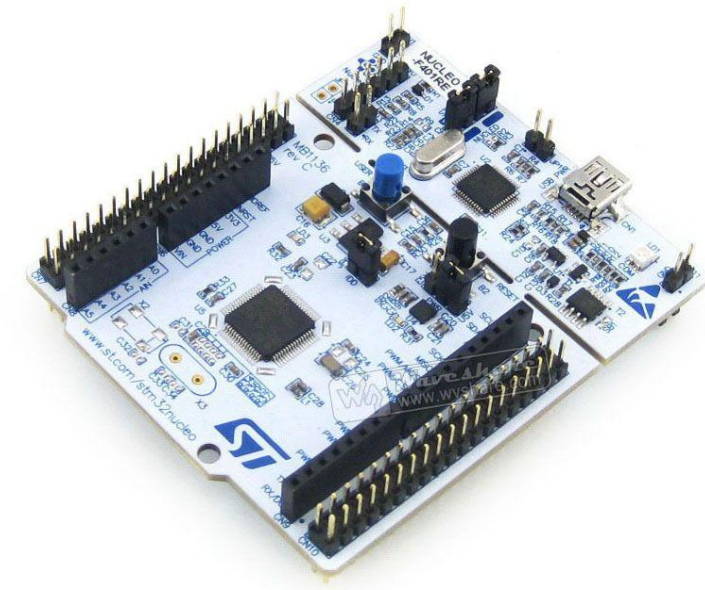
XXX Arduino connector names (A0, D1, ...)

XXX LEDs and Buttons (LED_1, USER_BUTTON, ...)

XXX AnalogIn (ADC) and AnalogOut pins (DAC)

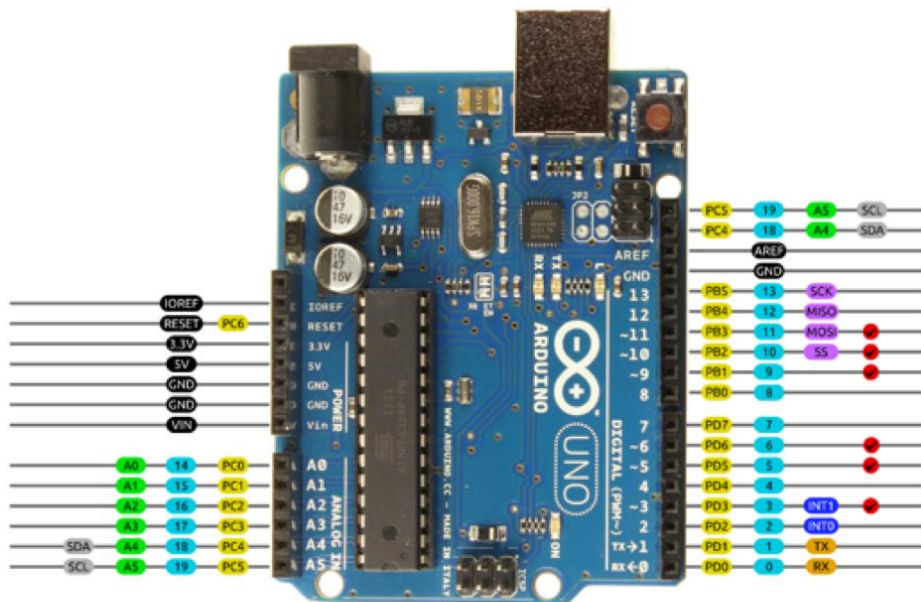
XXX CAN pins

XXX Power and control pins (3V3, GND, RESET, ...)

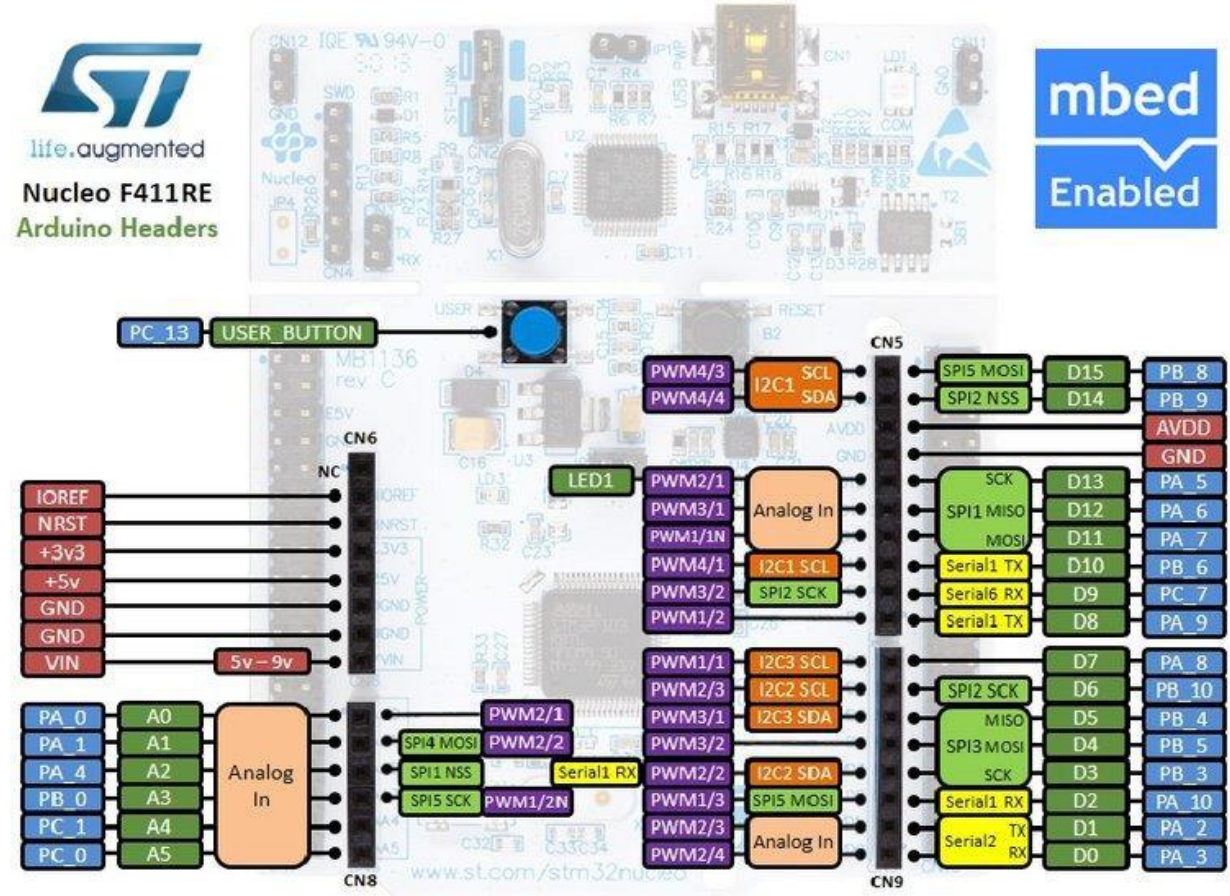


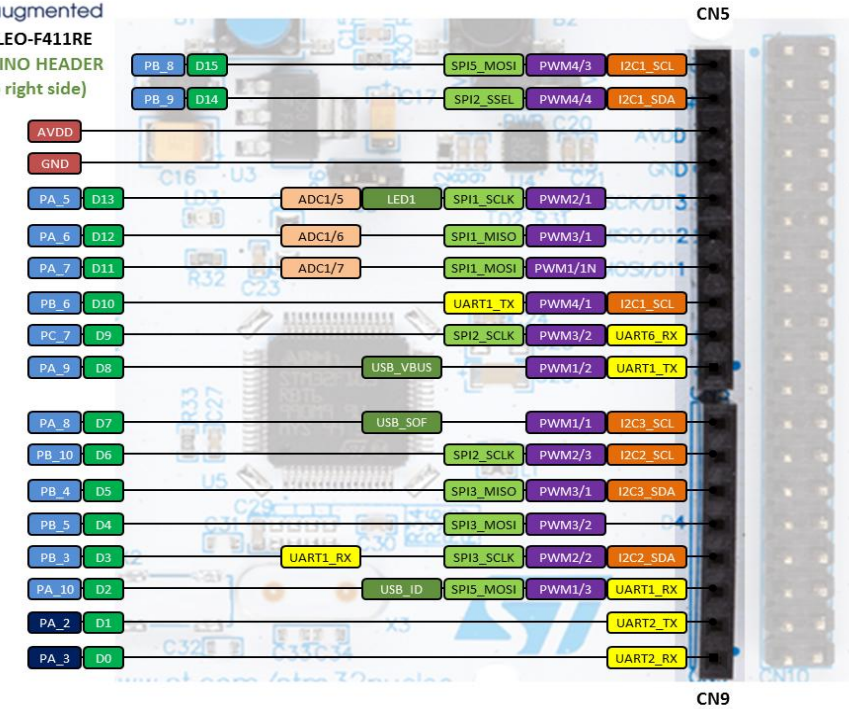
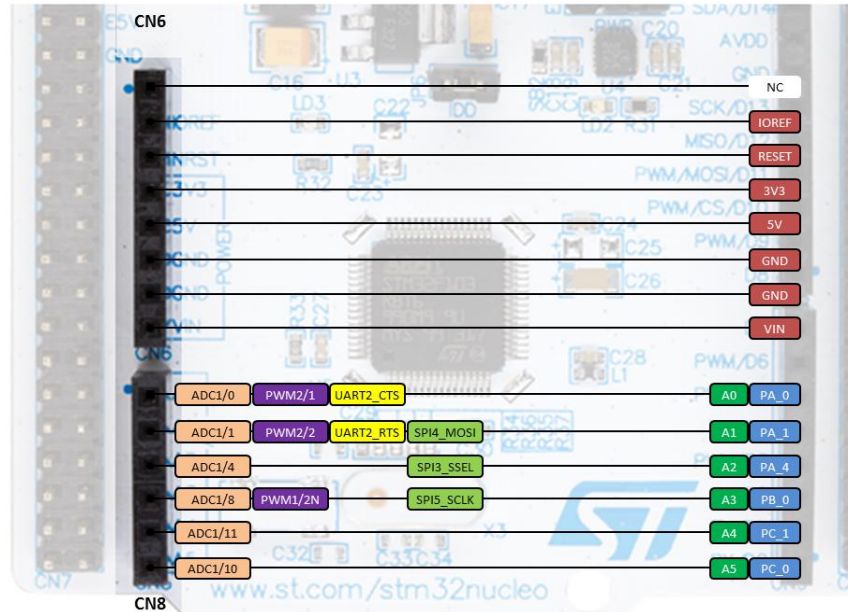
Comparing Arduino and Nucleo Board

Arduino Uno R3 Pinout



AVR DIGITAL ANALOG POWER SERIAL SPI I2C PWM INTERRUPT



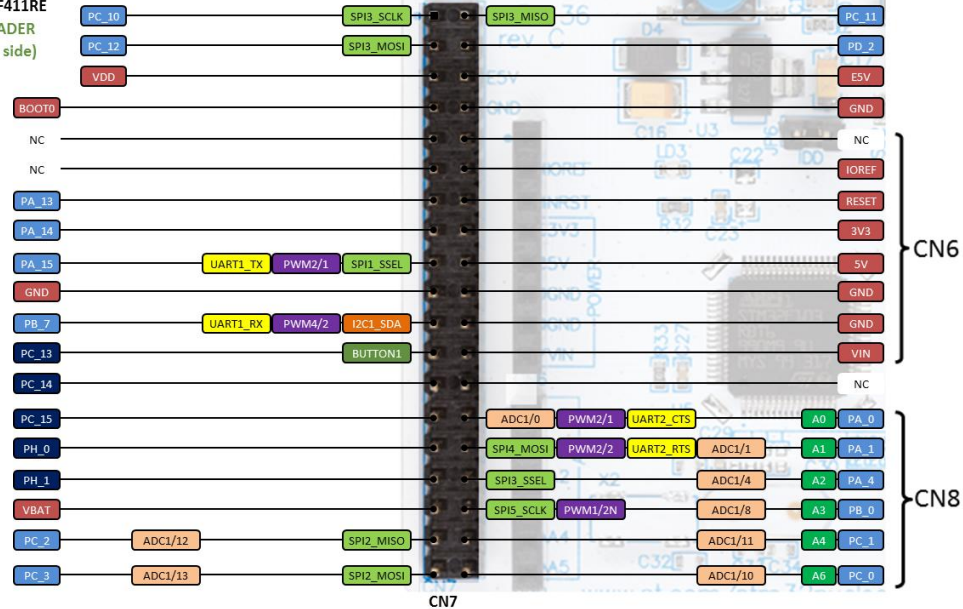


Arduino-compatible headers

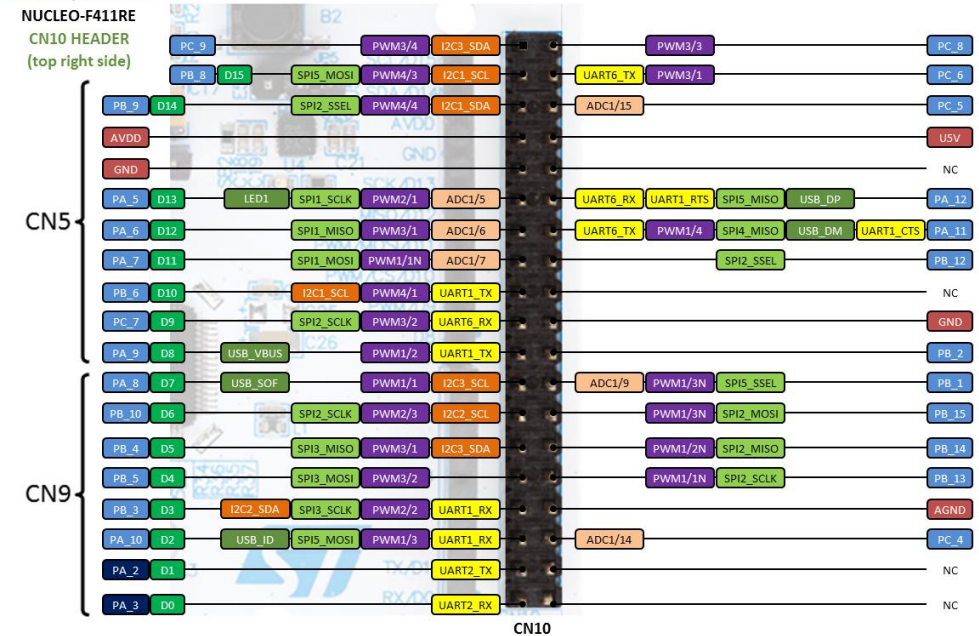
Morpho headers



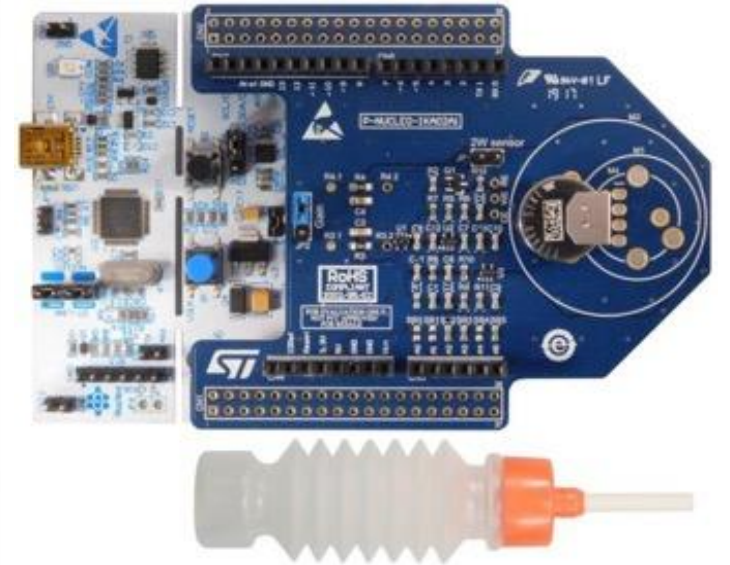
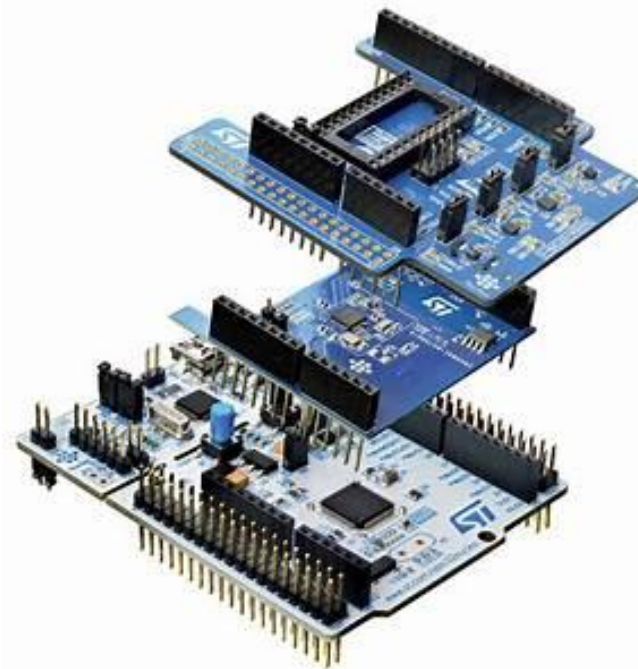
NUCLEO-F411RE
CN7 HEADER
(top left side)



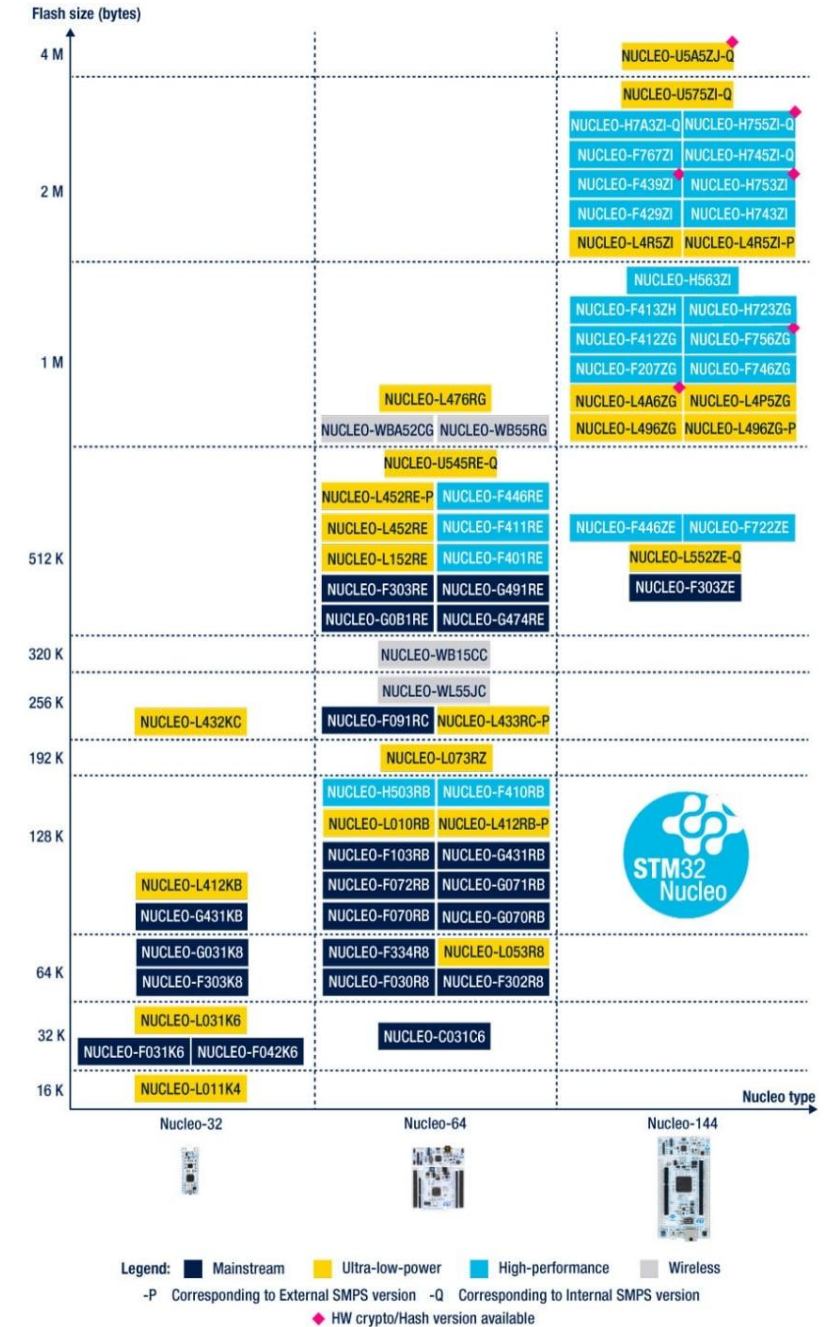
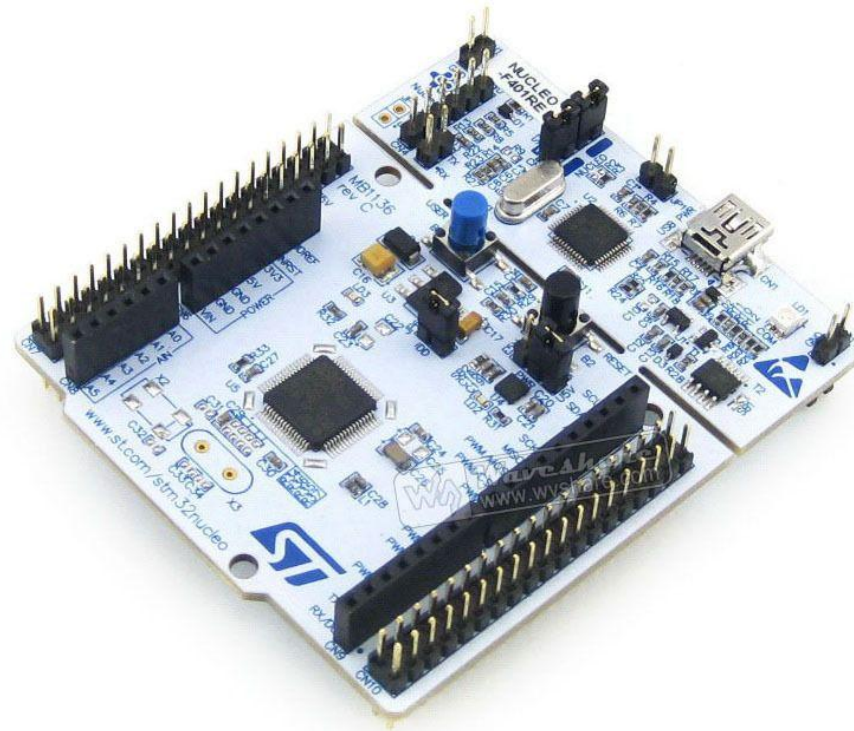
NUCLEO-F411RE
CN10 HEADER
(top right side)

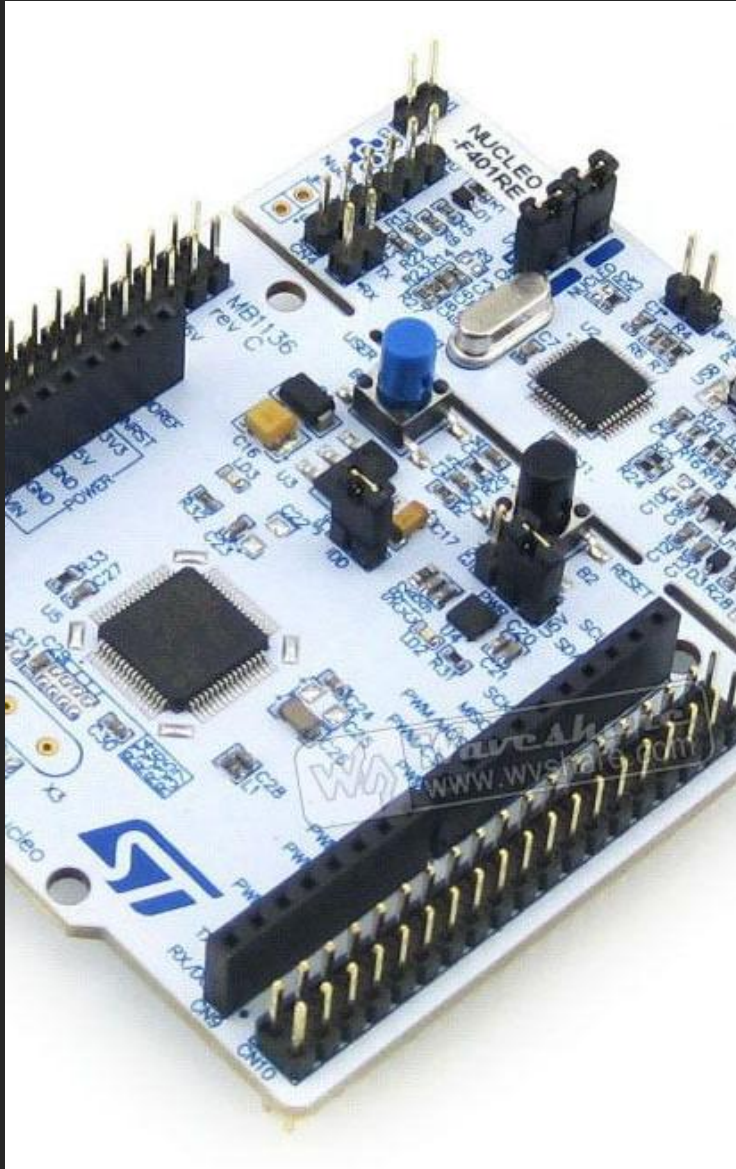


Expansion Shields for Nucleo Board



STM32 Nucleo Ecosystem

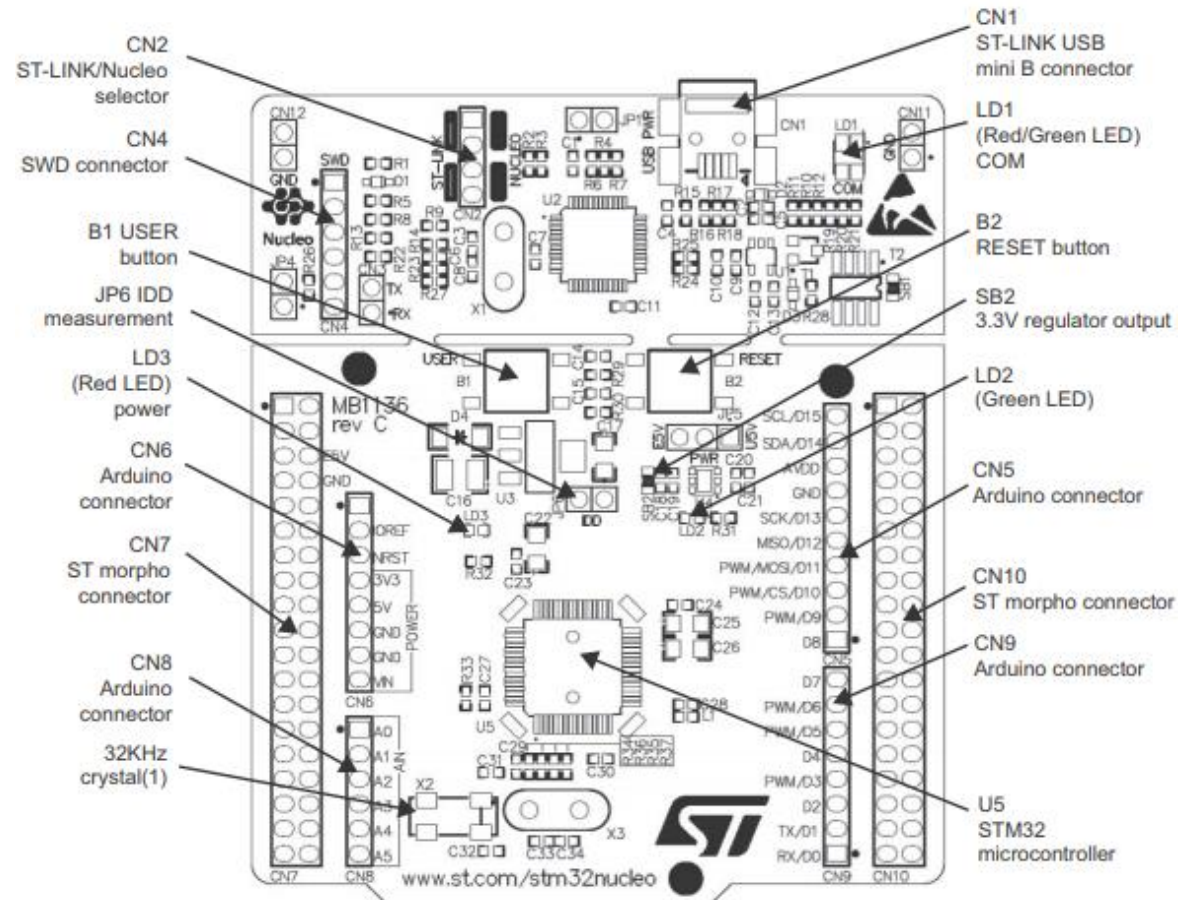




STM32F411RE MCU – STM32F411RET6

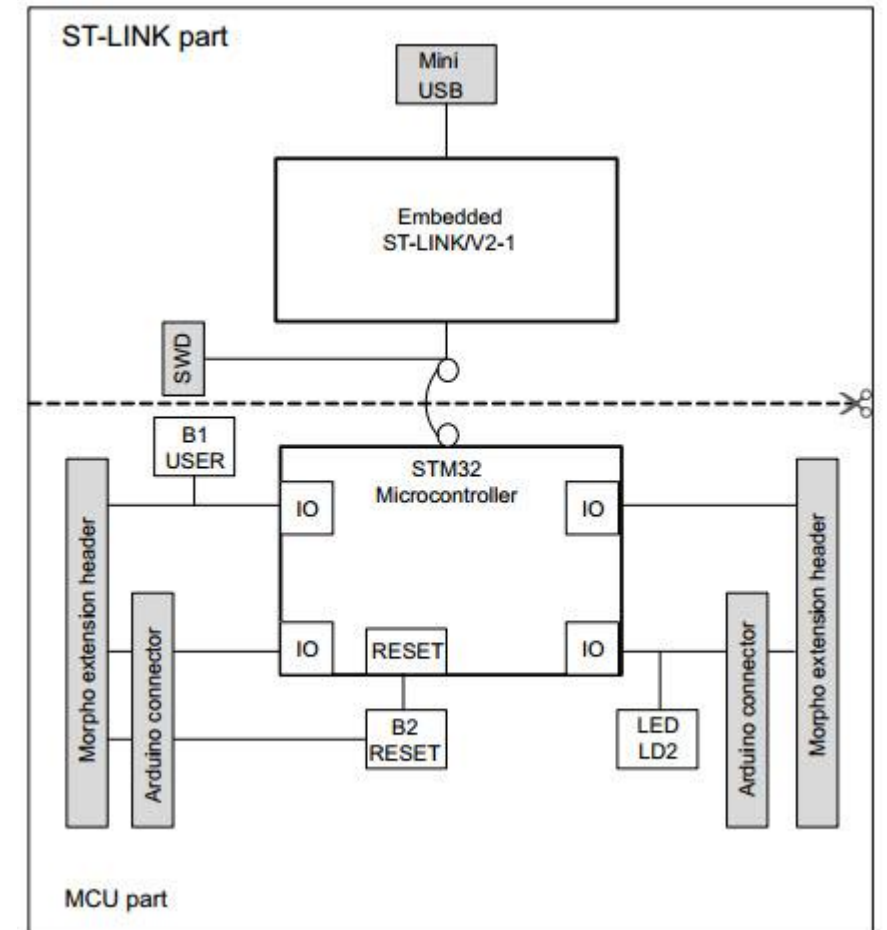
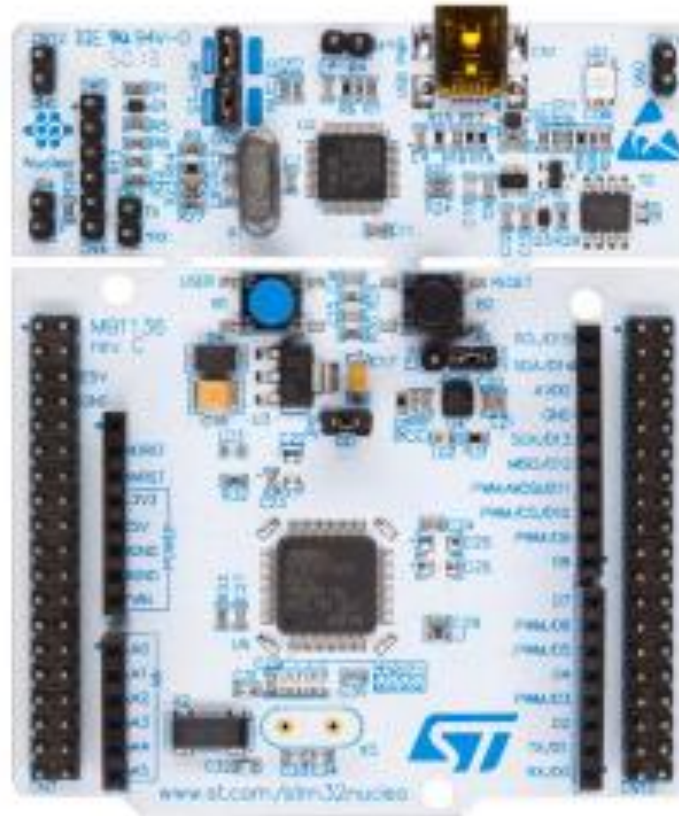
- STM32F411RET6 in LQFP64 package
- ARM®32-bit Cortex®-M4 CPU with FPU
- 100 MHz max CPU frequency
- VDD from 1.7 V to 3.6 V
- 512 KB Flash and 128 KB SRAM
- GPIO (50) with external interrupt capability
- 12-bit ADC with 16 channels
- RTC and Timers (8)
- I2C (3) - USART (3) - SPI (5)
- USB OTG Full Speed
- SDIO

STM32F411RE MCU Zooming in

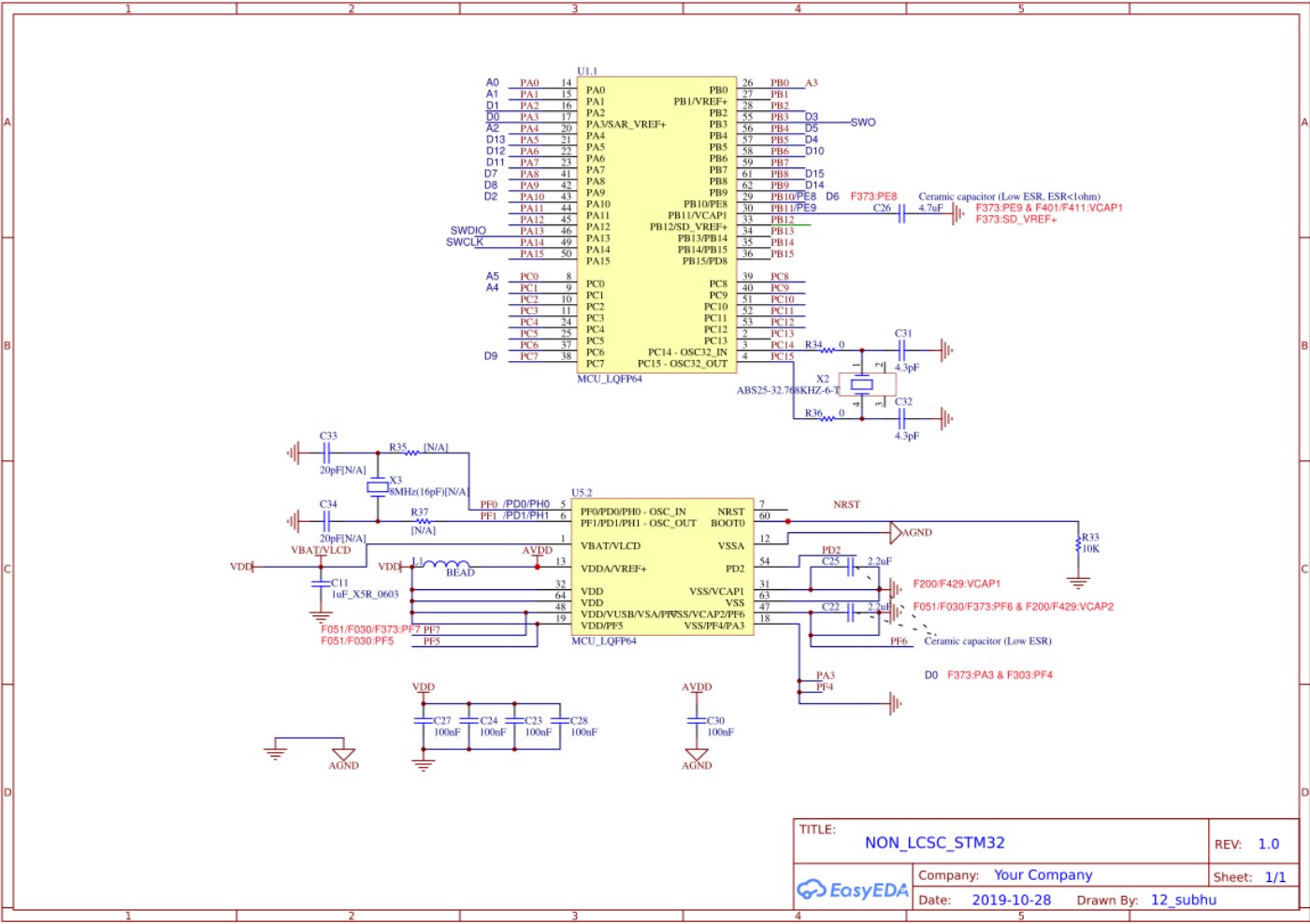


System View

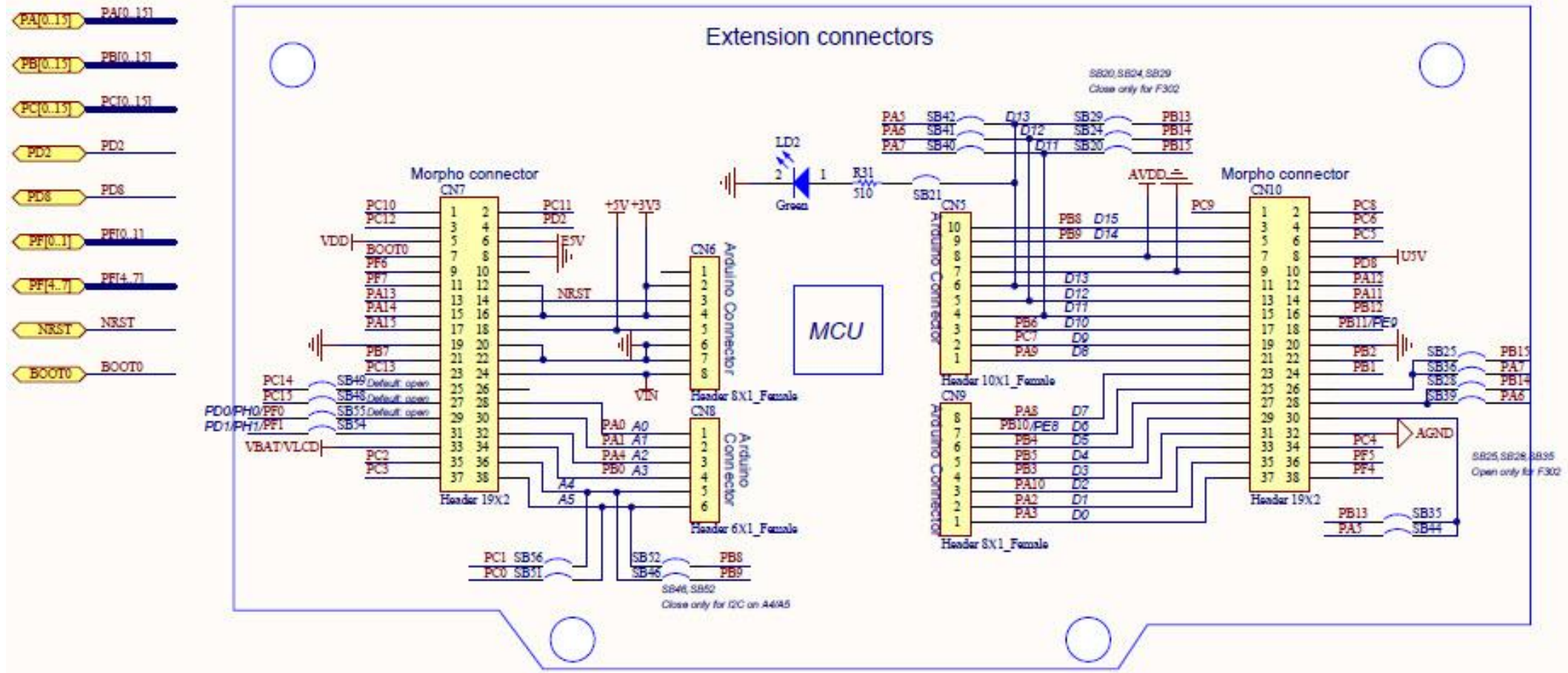
- The ST-LINK allows us to program the board



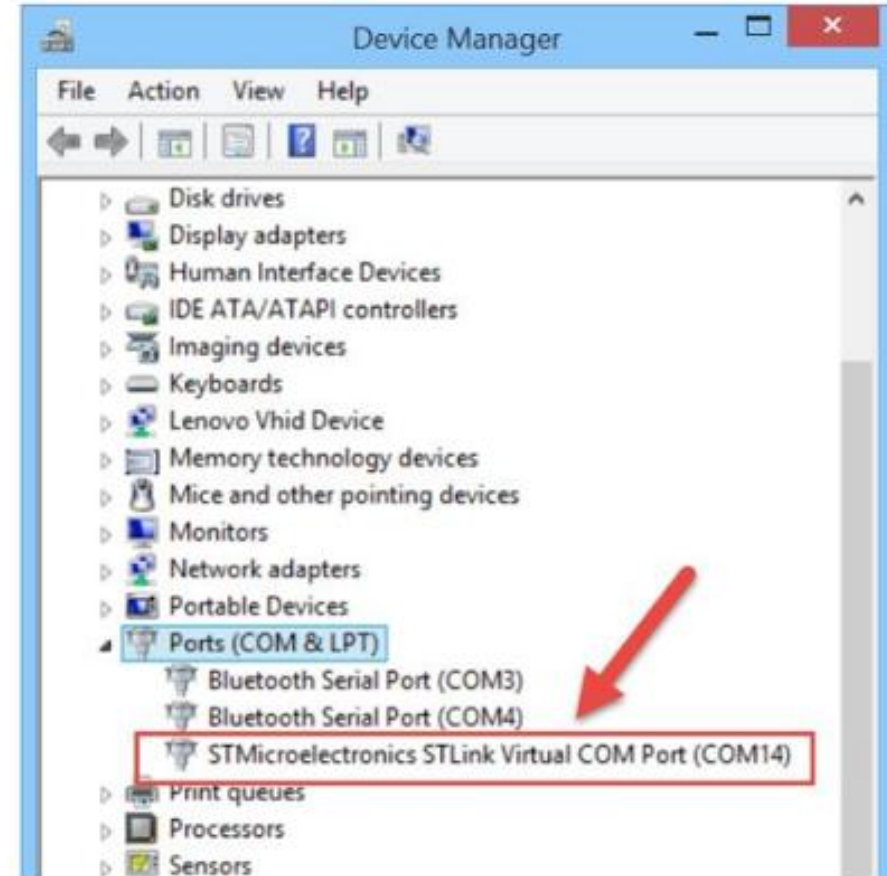
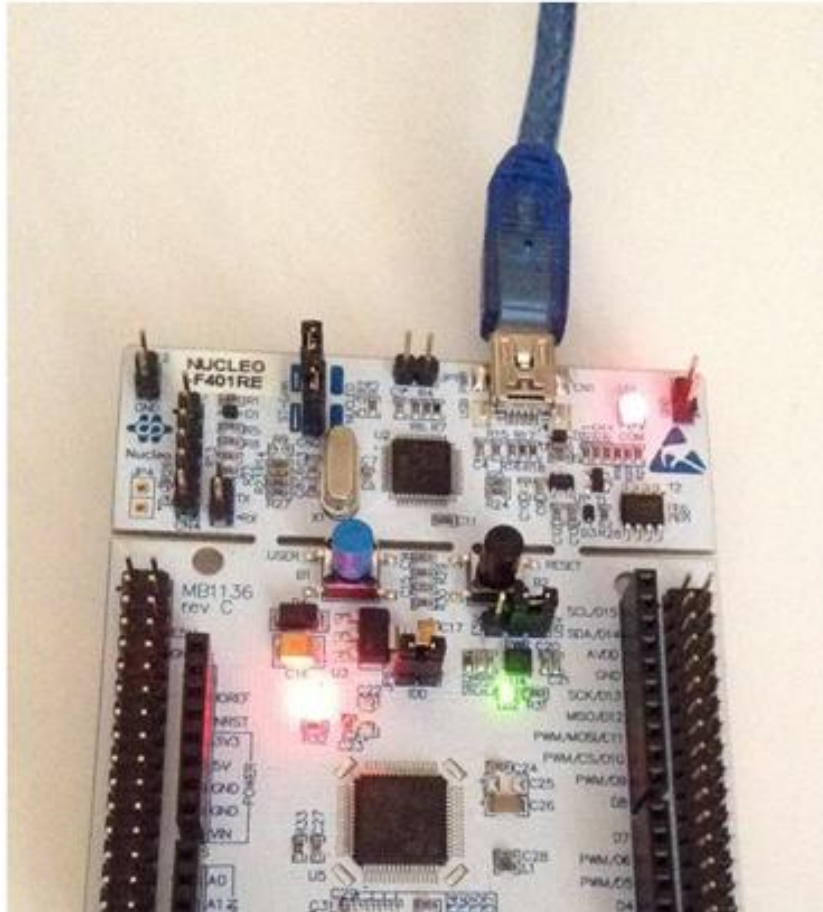
Board
Schematic



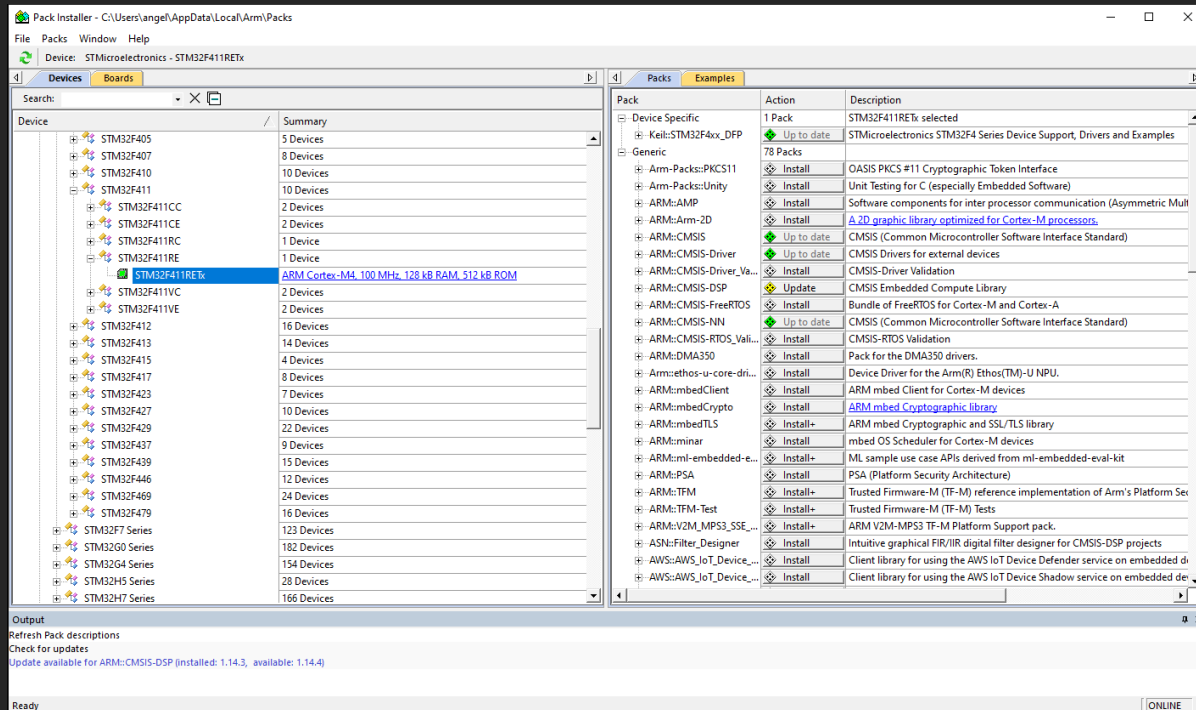
Extension Connectors



Checking your Connection



Start a NEW project in your Keil



- On the left side of the window under Devices, click to expand STMicroelectronics.
- Find STM32F4 Series and expand it
- Double click on STM32F411RE and it should add it to the packs list on the right side.
- Click all the buttons that say Install or Update to get all the features/packages that Keil offers.

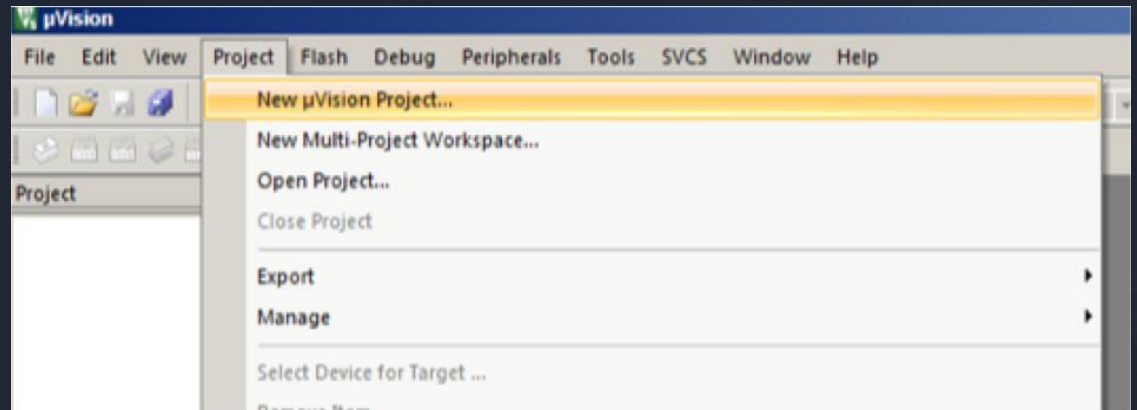
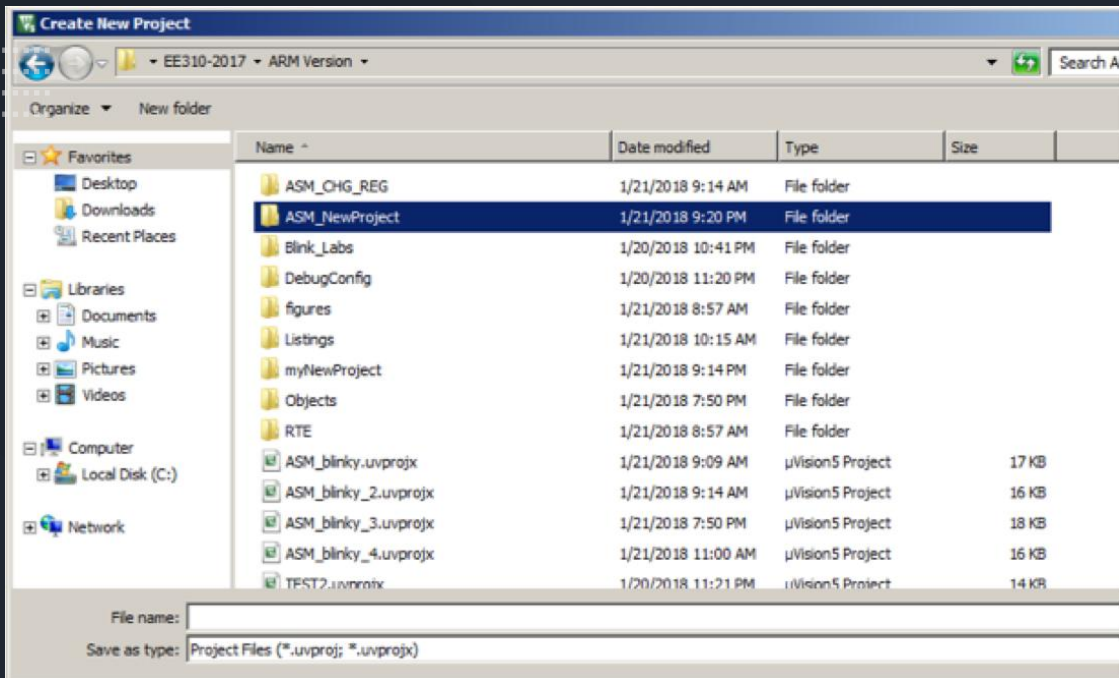
IMPORTANT!

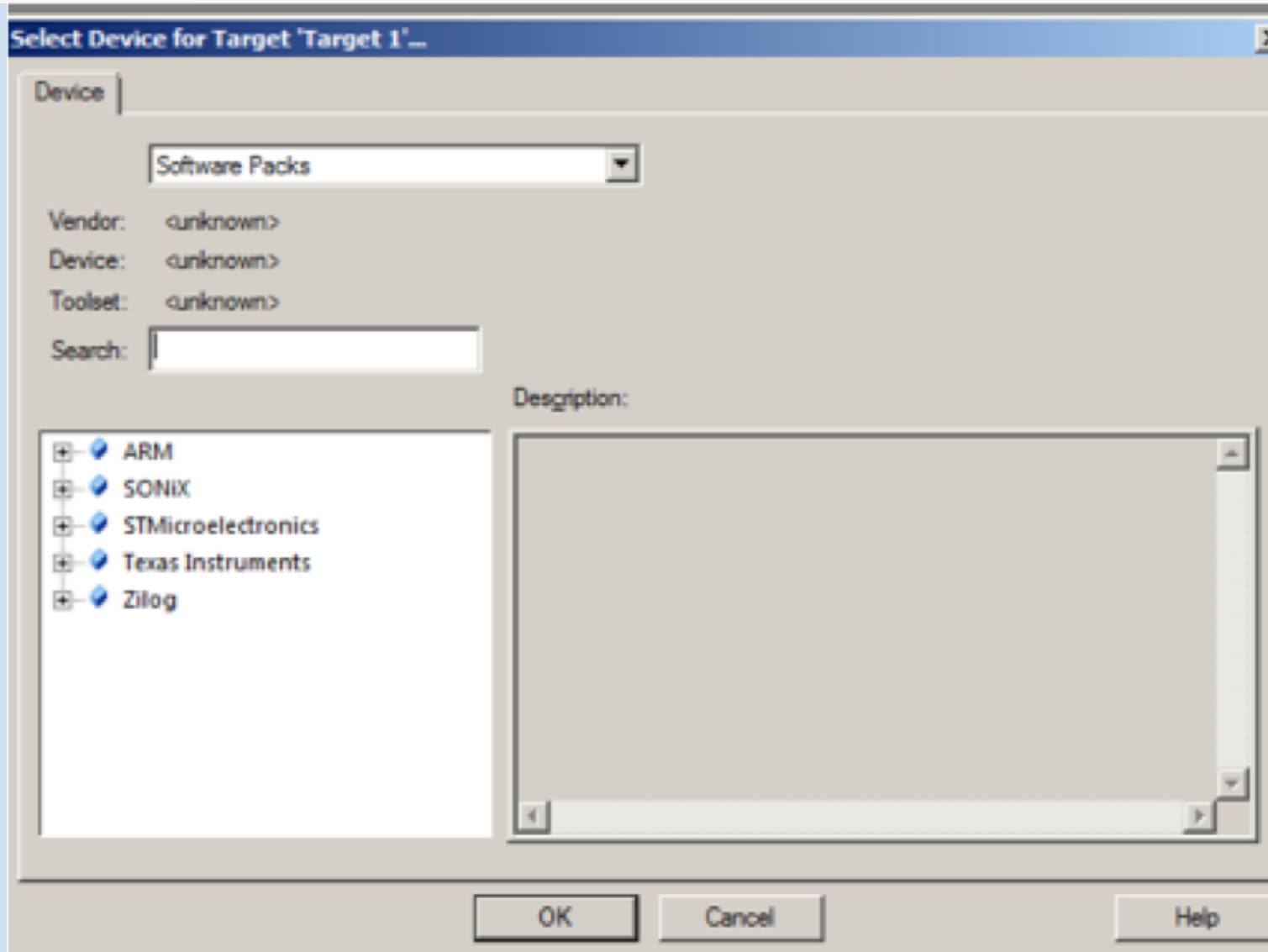
Use the 2.17.0 version of the DFP

Pack	Action	Description
[-] Device Specific	1 Pack	STM32F411 selected
[-] Keil::STM32F4xx_DFP	 Update	STMicroelectronics STM32F4 Series Device Support
3.0.0 (2024-10-11)	 Unpack	STMicroelectronics STM32F4 Series Device Support
2.17.0 (2023-01-16)	 Remove	STMicroelectronics STM32F4 Series Device Support, Drivers and Examples
2.16.0 (2022-01-19)	 Remove	STMicroelectronics STM32F4 Series Device Support, Drivers and Examples
+ Previous		Keil::STM32F4xx_DFP - Previous Pack Versions
[-] Generic	419 Packs	
+ AlifSemiconductor::AIPL	 Install+	Alif Semiconductor image processing library that uses D/AVE2D and Helium accel
AlifSemiconductor::As	 Install	Asynchronous support for Alif Semiconductor M55 LP and M55 LP device

Start A New Project

- Open Keil uVision 5 from your Windows
- Make sure all projects are closed:
- Project -> Close Project
- Click on Project -> new uVision Project ...
- In the window click on New Folder
- Type the name of the project:
ASM_NewProject
- Go to the new folder that you just created



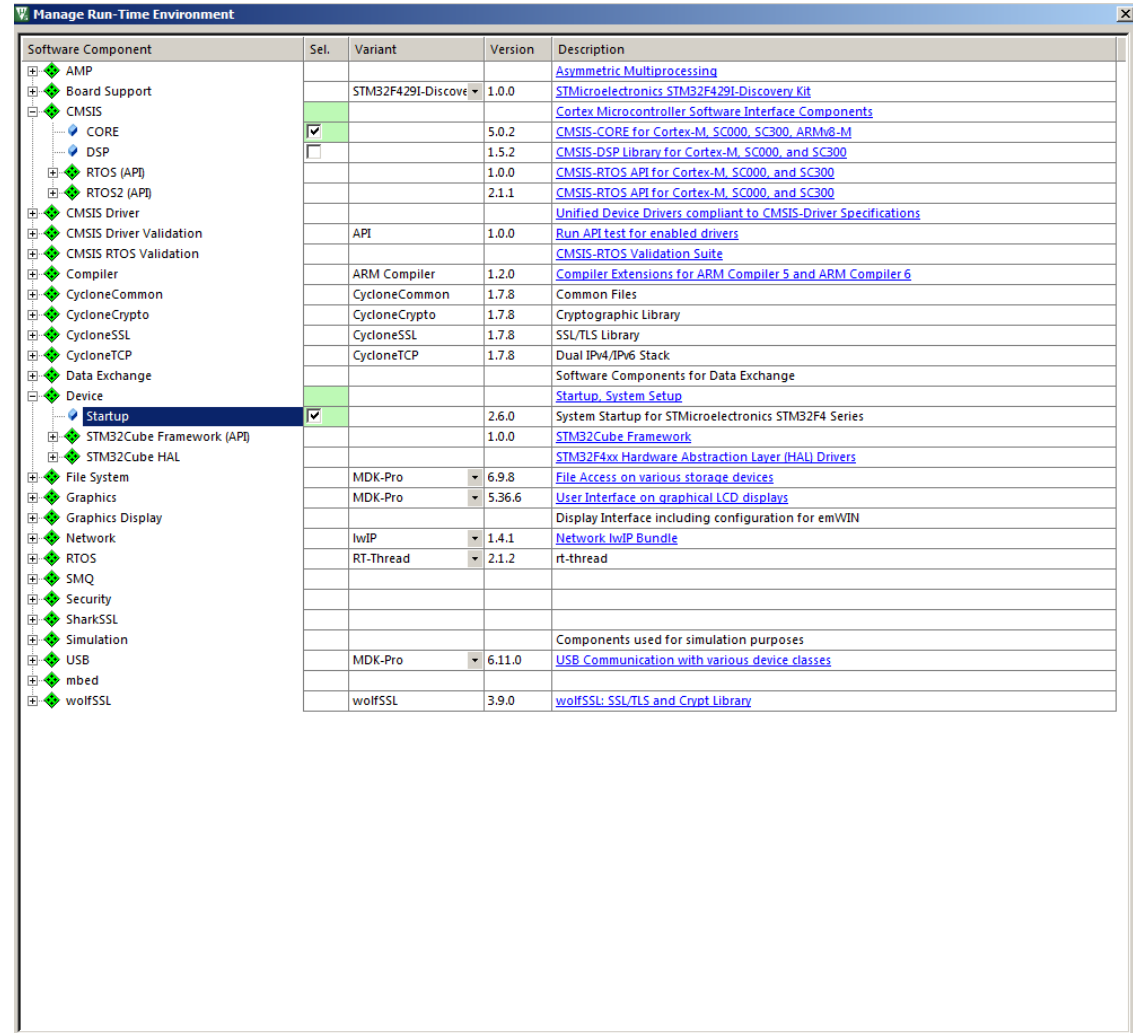


Start A New Project

- The Select Device window pops up
- In the search type: STM32F411RETx
- Click on the device (STM32F411RETx) that showed up and then click OK

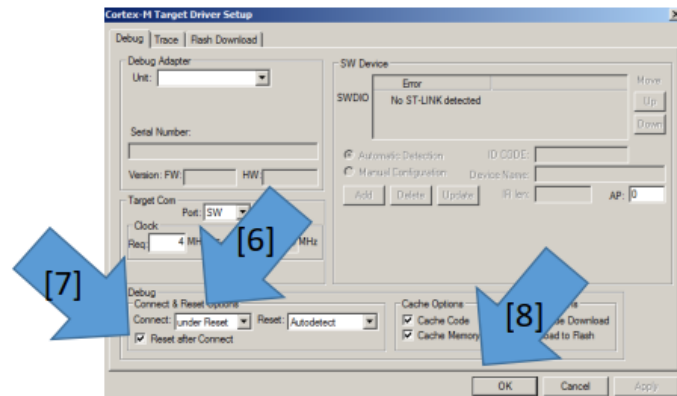
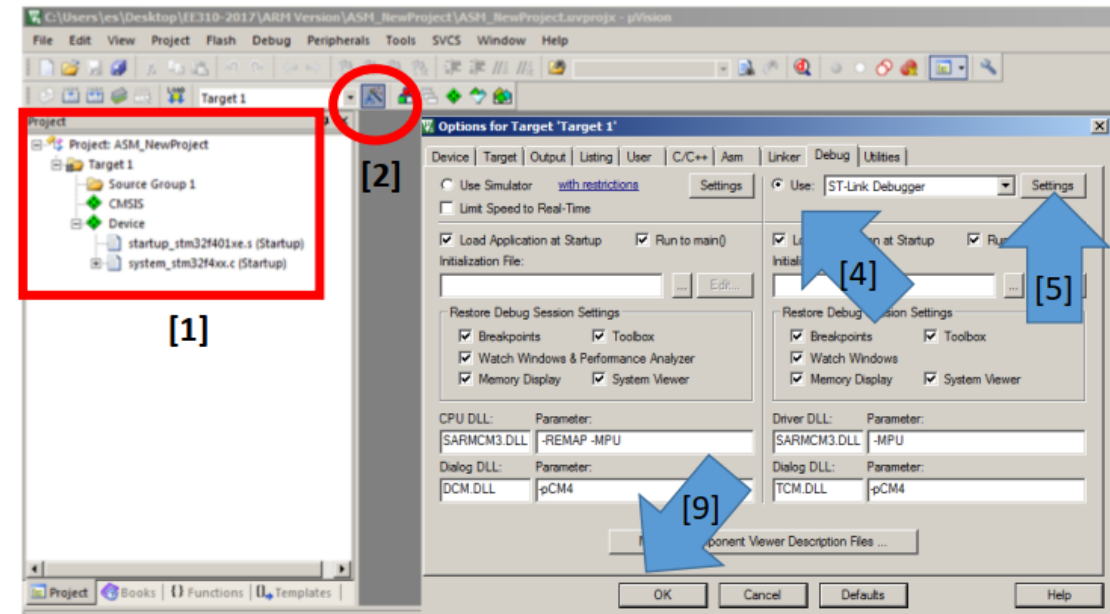
Start A New Project

- At this point the Manage Run window pops up
- CHECK CMSIS -> CORE and DEVICE->STARTUP boxes – note that the color changes
- Then click OK



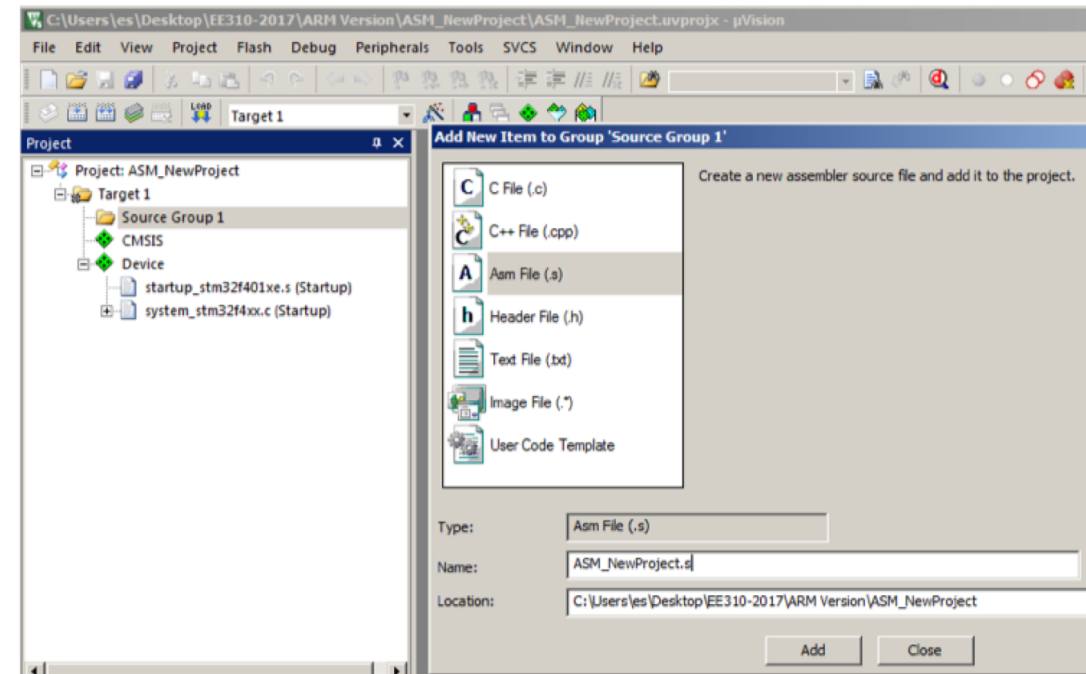
Start A New Project

- At this point your Project window should show the CMSIS and DEVICE – these are the CMSIS system initialization files and startup files in order for the RUN TIME to operate [1]
 - Click on the OPTIONS FOR TARGET [2]
 - Click on DEBUG tab [3]
 - CHECK and Select the USE ST-LINK [4]
 - **Click on Setting [5]**
 - In the CORTEX-M4 Target Drive
 - Choose Under Reset [6]
 - Check RESET AFTER CONTROL Box [7]
 - Click ok [8]
 - In the OPTIONS window, Click on OK [9]
 - In the PROJECT window click on Source Group 1 and select ADD NEW ITEM....
 - In the new window select



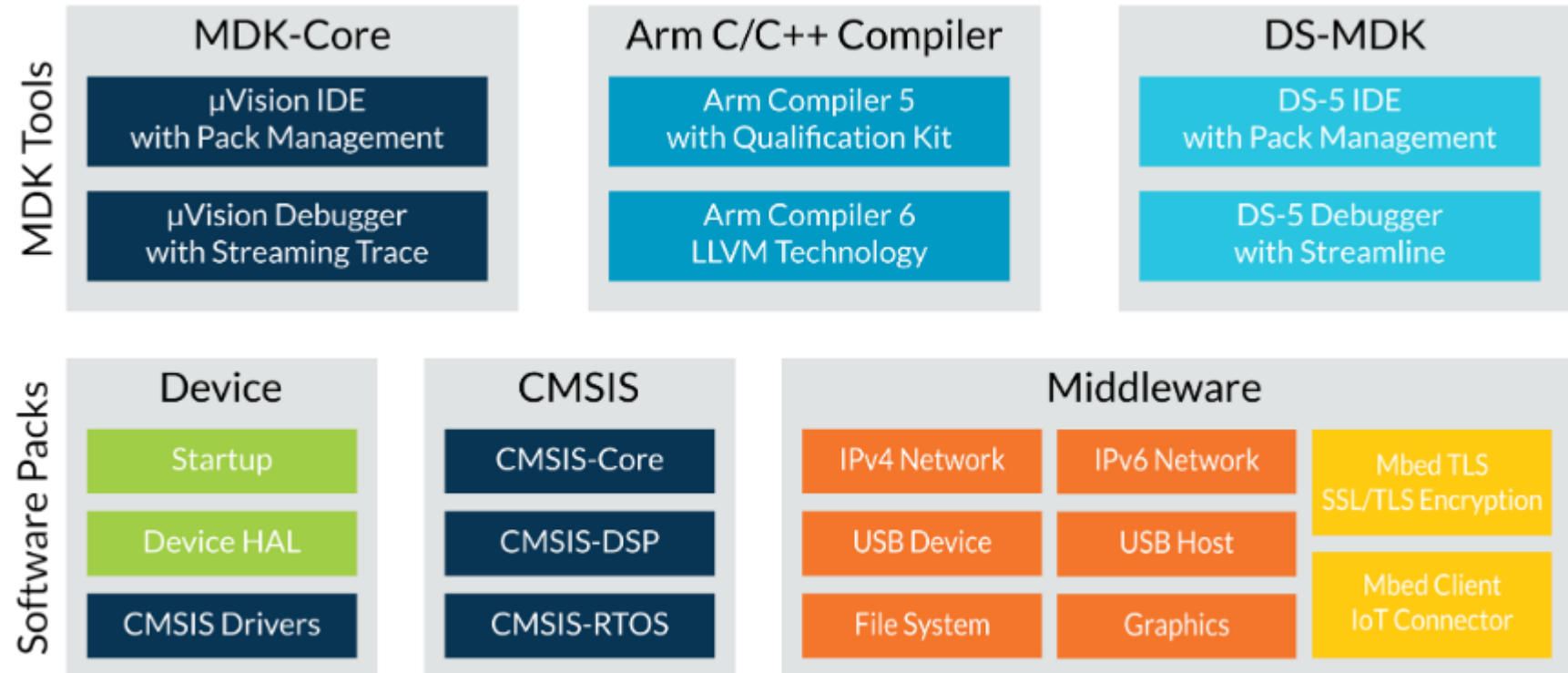
Start A New Project

- In the PROJECT window click on Source Group 1 and select ADD NEW ITEM....
- In the new window select ASM files
- Then type the name of the ASM file:
ASM_NewProject.s
- Click on ADD



Arm Keil MDK

Arm® Keil® MDK is the most comprehensive software development solution for Arm Cortex®-based microcontrollers. Every MDK edition includes IDE, C/C++ compiler, debugger, software pack management, and CMSIS



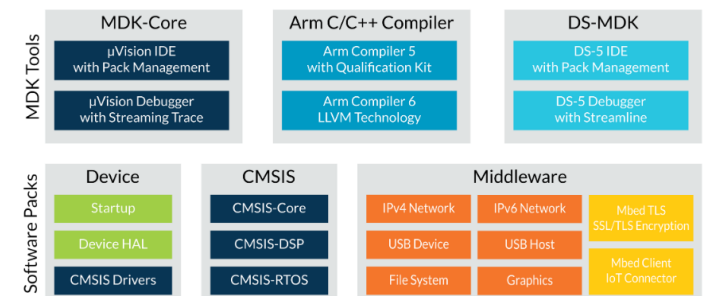
Arm Keil MDK

MDK-Core offers leading support for over 3,750 Cortex-M based devices including the new Arm Cortex-M23/M33 cores. The μ Vision debugger includes event recorder and component viewer to show run-time behavior of software components. Together with the ULINKpro™ debug adapter, it offers full instruction trace and complete code coverage information.

DS-MDK contains the Eclipse-based DS-5 IDE/debugger and supports 32-bit Cortex-A processors or hybrid systems with 32-bit Cortex-A and Cortex-M. Streamline performance analyzer helps to get the best out of the system's resources and create high performance, energy efficient products.

Software packs can be added any time to MDK-Core or DS-MDK making new device support and middleware updates independent from the toolchain. They contain device support, CMSIS libraries, software components, middleware, board support, code templates, and example projects.

MDK middleware provides royalty-free, tightly-coupled software components that are specifically designed for communication peripherals in microcontrollers.

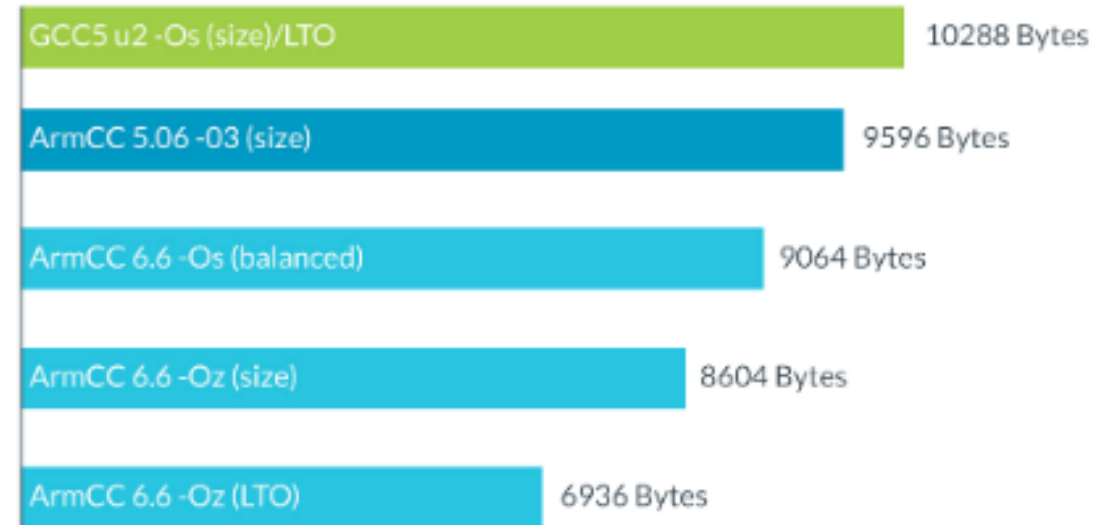


Arm Keil MDK

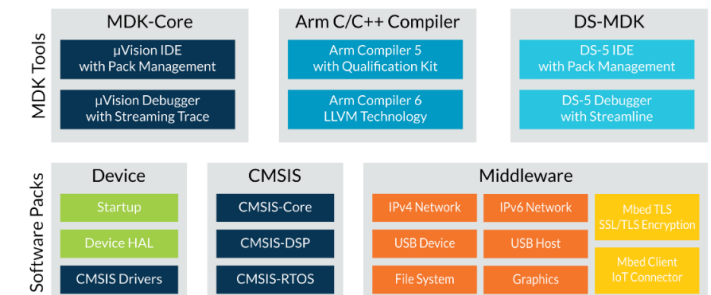
The industry-leading Arm C/C++ Compilers with assembler, linker, and highly optimized run-time libraries are tailored for optimum code size and performance.

Arm Compiler 5 is certified for functional safety applications and offers long-term maintenance and support. Arm Compiler 6 offer the best code size currently on the market.

It offers various optimization levels including Link Time Optimization. This diagram shows the code size.



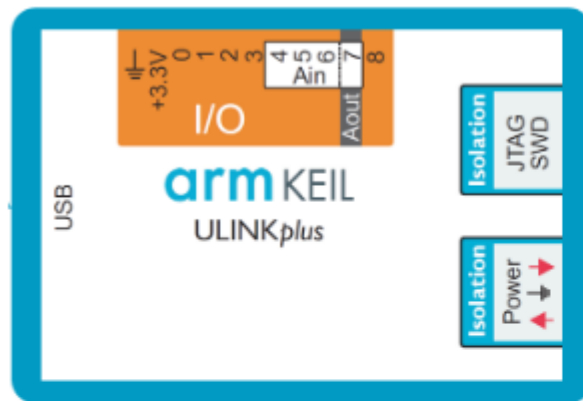
developer.arm.com/armcompiler6



MDK editions and ULINK debug adapters

	Professional	Plus	Essential	Lite
µVision				
IDE with Editor, Pack Installer	✓	✓	✓	✓
Debugger	✓	✓	✓	32 KB
Fixed Virtual Platforms Simulation Models	✓			
DS-MDK (Windows® host + Linux® host)				
Support for heterogenous systems (Linux Cortex-A, RTOS Cortex-M)	✓	✓		
Streamline system analysis	✓			
Arm Compiler				
C/C++ Compilation Tools	✓	✓	✓	32 KB
Extended Maintenance and Qualification Kit	✓			
Arm processor support				
Arm Cortex-M0 .. Cortex-M7	✓	✓	✓	✓
Arm Cortex-M23/33 Non-Secure	✓	✓	✓	
Arm Cortex-M23/33 Secure + Non-Secure	✓	✓		
Armv8-M architecture + FastModel	✓			
Arm SecurCore™ (SC000, SC300)	✓	✓		
Arm7, Arm9, Arm Cortex-R4	✓	✓		
RTOS and Middleware				
CMSIS-RTOS RTX with full source code	✓	✓	✓	✓
Middleware (IPv4 Network, USB Device, File System, Graphics)	✓	✓		
Middleware (IPv6 Network, USB Host, IoT Connectivity)	✓			

MDK editions and ULINK debug adapters

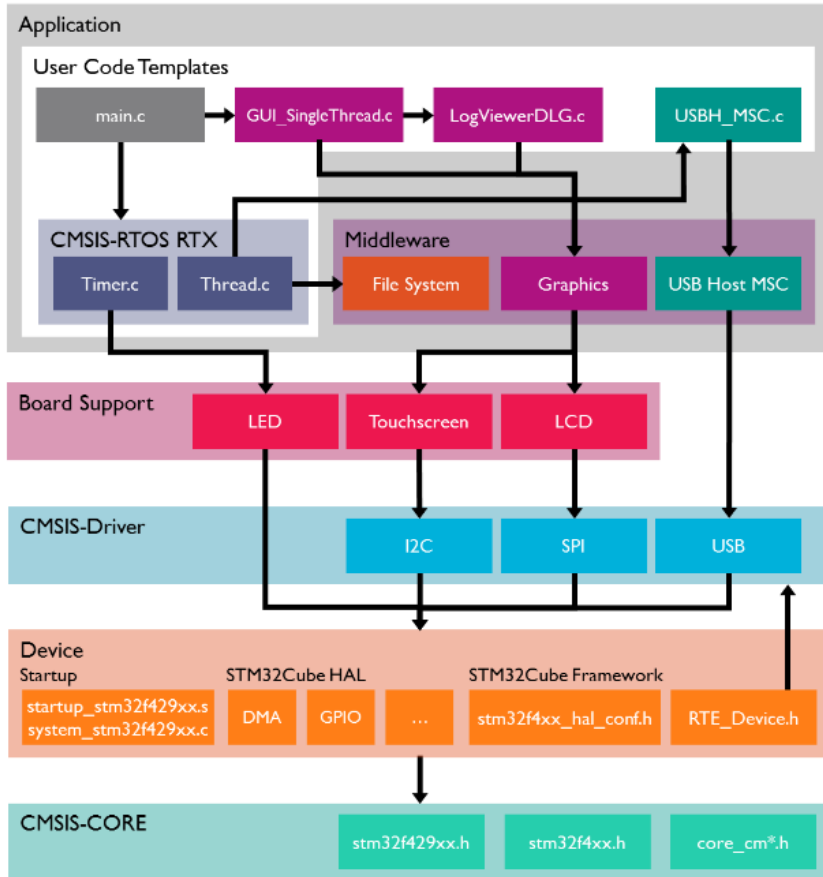


ULINKplus is a universal debug and trace adapter that enables test automation, software optimization for ultra-low-power applications, and isolation for debugging sensitive hardware systems. The compact enclosure allows usage in harsh environments and provides standard target connectors for JTAG, power measurement, and general purpose I/O.

Features	ULINKpro	ULINKplus	ULINK2
Run control	✓	✓	✓
Memory and breakpoint	✓	✓	✓
Data trace	✓	✓	✓
Instruction trace	✓		
Performance			
JTAG clock speed	50 MHz	50 MHz	50 MHz
Memory read/write	1 MB/s	3 MB/s	25 KB/s
Data and event trace	100 Mbit/s	50 Mbit/s	1 Mbit/s
Instruction trace	800 Mbit/s		
Analysis tools			
Component viewer	✓	✓	✓
Event recorder	✓	✓	✓
Power measurement		✓	
General purpose I/Os		✓	
Performance analyzer	✓		
Execution profiler	✓		
Code coverage	✓		

www.keil.com/ulink

CMSIS Components

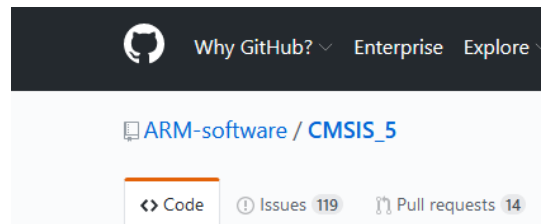


The **Cortex Microcontroller Software Interface Standard (CMSIS)** is a vendor-independent hardware abstraction layer for microcontrollers that are based on Arm® Cortex® processors. It defines generic tool interfaces and enables consistent device support. Its software interfaces simplify software re-use, reduce the learning curve for microcontroller developers, and improve time to market for new devices. CMSIS provides interfaces to processor and peripherals, real-time operating systems, and middleware components. It includes a delivery mechanism for devices, boards, and software and enables the combination of software components from multiple vendors.



CMSIS Components

CMSIS-...	Target Processors	Description
Core(M)	All Cortex-M, SecurCore	Standardized API for the Cortex-M processor core and peripherals. Includes intrinsic functions for Cortex-M4/M7/M33/M35P SIMD instructions.
Core(A)	Cortex-A5/A7/A9	Standardized API and basic run-time system for the Cortex-A5/A7/A9 processor core and peripherals.
Driver	All Cortex	Generic peripheral driver interfaces for middleware. Connects microcontroller peripherals with middleware that implements for example communication stacks, file systems, or graphic user interfaces.
DSP	All Cortex-M	DSP library collection with over 60 Functions for various data types: fixed-point (fractional q7, q15, q31) and single precision floating-point (32-bit). Implementations optimized for the SIMD instruction set are available for Cortex-M4/M7/M33/M35P.
NN	All Cortex-M	Collection of efficient neural network kernels developed to maximize the performance and minimize the memory footprint on Cortex-M processor cores.
RTOS v1	Cortex-M0/M0+/M3/M4/M7	Common API for real-time operating systems along with a reference implementation based on RTX. It enables software components that can work across multiple RTOS systems.
RTOS v2	All Cortex-M, Cortex-A5/A7/A9	Extends CMSIS-RTOS v1 with Armv8-M support, dynamic object creation, provisions for multi-core systems, binary compatible interface.
Pack	All Cortex-M, SecurCore, Cortex-A5/A7/A9	Describes a delivery mechanism for software components, device parameters, and evaluation board support. It simplifies software re-use and product life-cycle management (PLM).
SVD	All Cortex-M, SecurCore	Peripheral description of a device that can be used to create peripheral awareness in debuggers or CMSIS-Core header files.
DAP	All Cortex	Firmware for a debug unit that interfaces to the CoreSight Debug Access Port.
Zone	All Cortex-M	Defines methods to describe system resources and to partition these resources into multiple projects and execution areas.



Tips: 1st Assignment

1. Create an assembly (.s) file in your source folder
2. In the assembly source, declare the code as a global function using .global and .type:

```
.global myadd
.p2align 2
.type myadd,%function

myadd:                // Function "myadd" entry point.
    .fnstart
    add    r0, r0, r1  // Function arguments are in R0 and R1. Add together
                        // and put the result in R0.
    bx     lr          // Return by branching to the address in the link
                        // register.
    .fnend
```

3. In C code, declare the external function using extern:

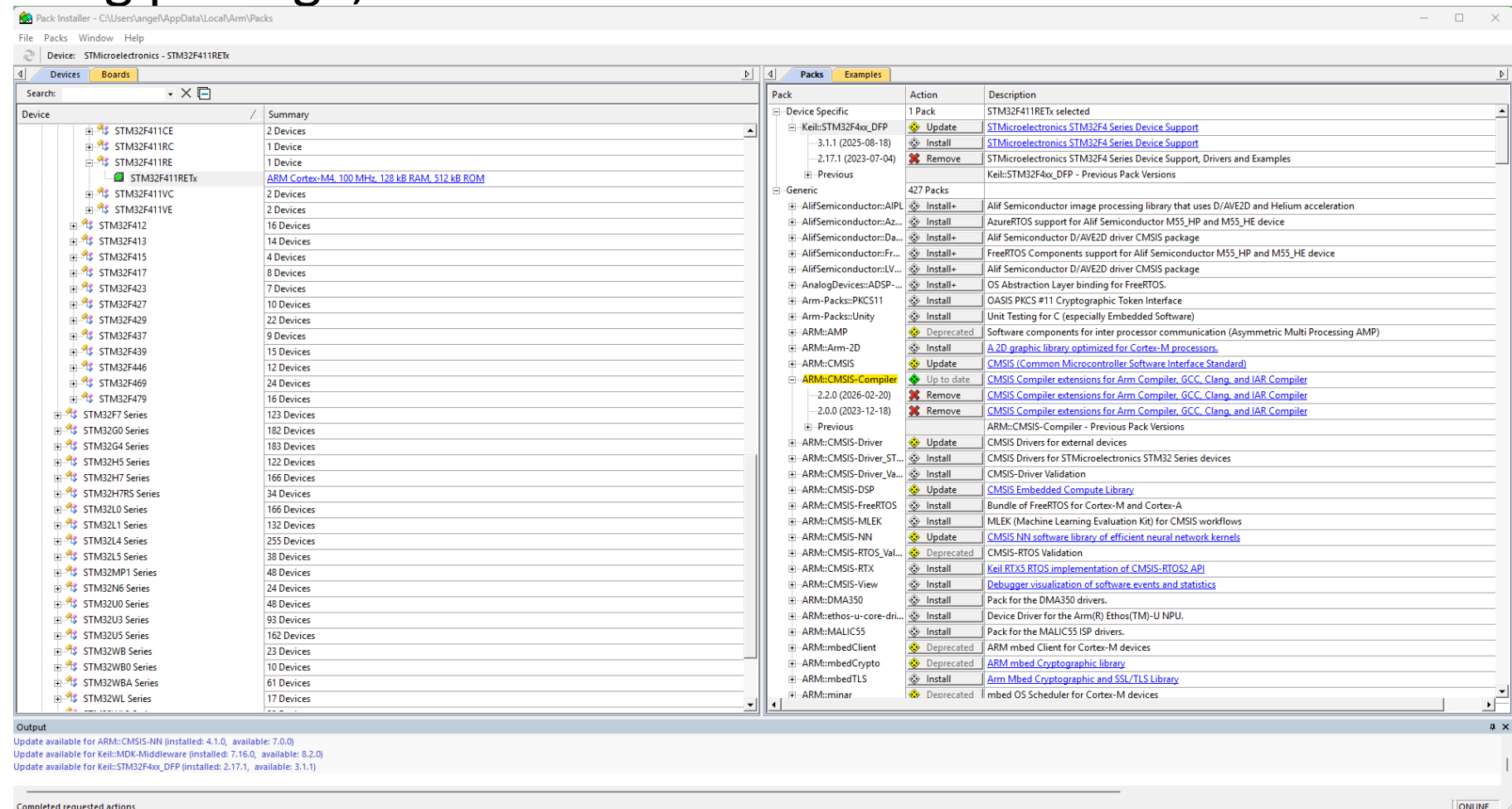
```
#include <stdio.h>

extern int myadd(int a, int b);

int main()
{
    int a = 4;
    int b = 5;
    printf("Adding %d and %d results in %d\n", a, b, myadd(a, b));
    return (0);
}
```

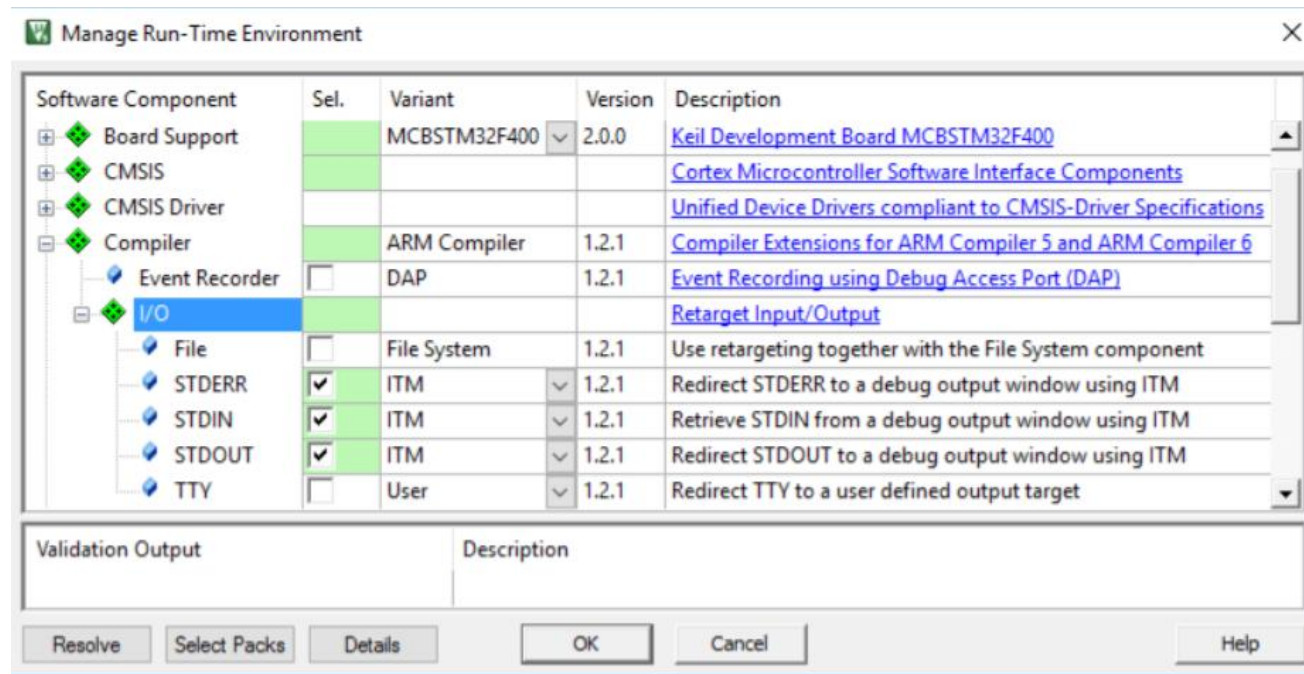

Tips: How to enable Keil-printf (1)

1. Install the following package, from Pack Installer



Tips: How to enable Keil-printf (2)

2. Open the Manage RTE dialog and expand the Compiler -- I/O folder
3. Check the STDERR, STDIN and STDOUT components and set the Variant to **ITM** for each component.

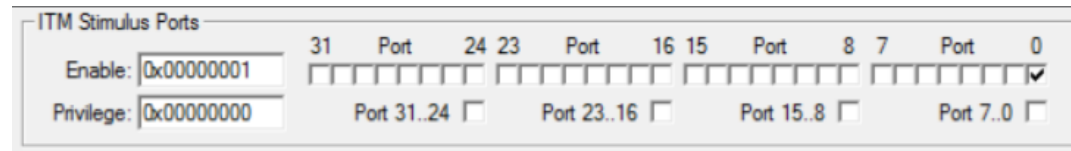


Tips: How to enable Keil-printf (3)

4. Add a debugging trace messages using printf

```
printf("AD value = 0x%04X\r\n", AD_value);
```

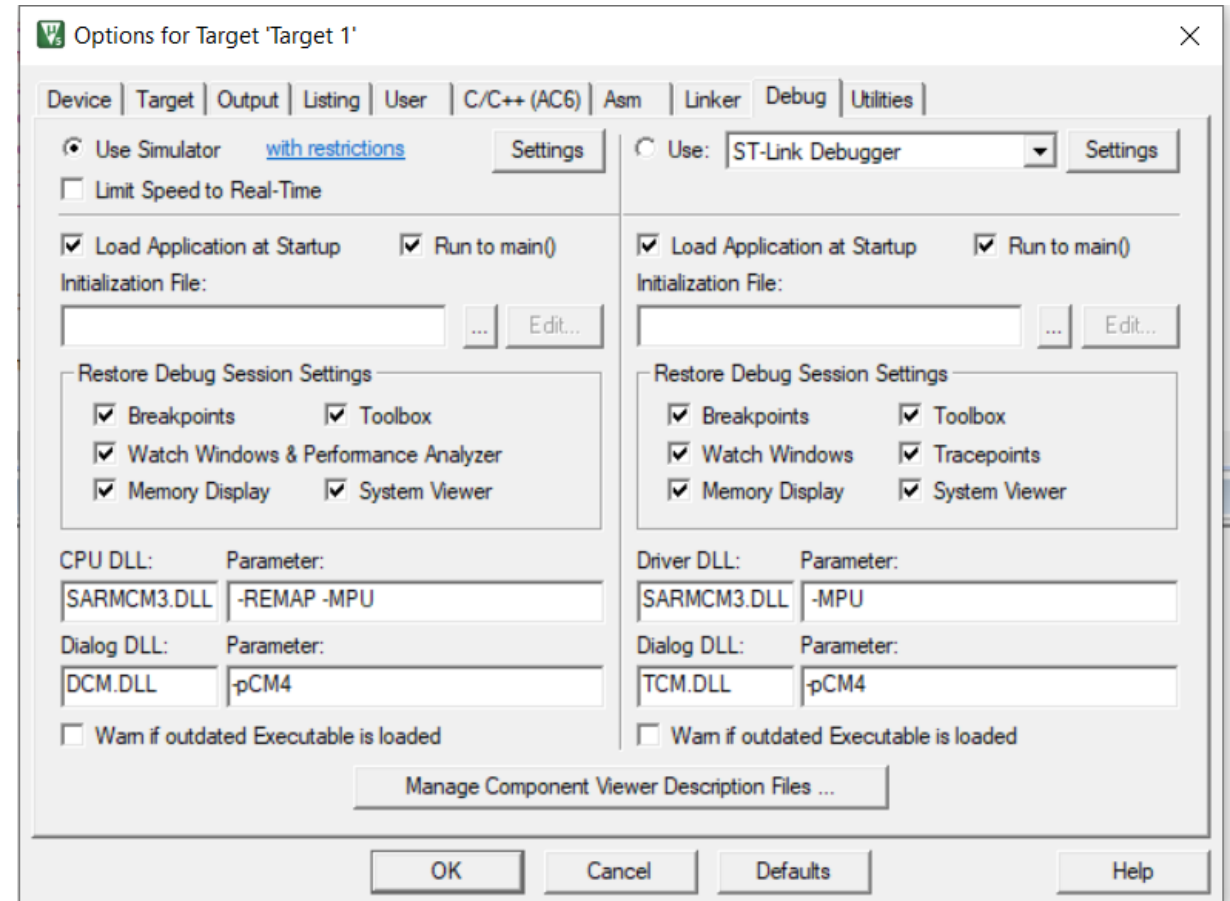
5. Set up the debug session for SWO trace
 - Trace Enable
 - Set Core Clock (usually 16MHz)
6. Set the ITM Port 0 to capture the information. Clear the Privilege Port 7..0 to access ITM Port 0 from User Mode.



7. Open the window from the menu View - Serial Windows - Debug (printf) Viewer.

Tips: How to enable Simulation

1. Click on Options for Target...
2. Click Debug
3. Enable the Use Simulator option



Download keil

<https://www.keil.com/download/product/>

armKEIL

ProductsDownloadEventsSupportVideos

Search Keil...

Go

Download Products

Select a product from the list below to download the latest version.

MDK

MDK-Arm
Version 5.39 (December 2023)
Development environment for Cortex and Arm devices.

C51

C51
Version 9.61 (December 2022)
Development tools for all 8051 devices.

C251

C251
Version 5.60 (May 2018)
Development tools for all 80251 devices.

C166

C166
Version 7.57 (May 2018)
Development tools for C166, XC166, & XC2000 MCUs.

Keil products use a [License Management](#) system - without a current license the product runs as a Lite/Evaluation edition with a few [Limitations](#).

Maintenance Status and Previous Versions

Enter a valid Product Serial Number (**PSN**) or License Code (**LIC**) to get access to all product versions available to you, or to check the status of your support and maintenance agreement.

PSN or LIC:

Further information about installing your software is available in the [Read Me First](#) brochure.

Note
For **Flex Floating licenses**, please use the PSN mentioned in [Knowledgebase Article 3698](#). Refer to the [FlexLM Client Setup Guide](#) for setup instructions.

Note

- Use a current Product Serial Number (PSN) to get a License Code (LIC) that converts the Lite/Evaluation version to a full edition
- Evaluation editions of Keil software products do not require a Product Serial Number or License Code.
- Vendor specific toolkits should be downloaded from <https://www.keil.com/demo/eval/arm.htm>